Chapter 1

INTRODUCTION

Due to the recent economic growth and the availability of low-priced cars in the market, every average middle-class individual can afford a car, which is a good thing. However, the consequences are heavy traffic jams, pollution, less availability of roads, and a lack of spots to park the motor car. One of the important concerns, which is to be taken into account, is the problem of parking those vehicles. Though there is space for parking the vehicle, so much time is wasted in finding that exact parking slot, resulting in more fuel consumption, which is not environmentally friendly. If in some way we find out the precise vacant position of the parking slot, then it would be helpful not only to the drivers but also to the environment. Therefore, many innovations are created to find solutions. This project also proposes one such smart parking solution. Here we describe a high-level view of the system architecture. Towards the end, the project discusses the workings of the system in the form of a use case that proves the correctness of the proposed model. The smart parking system that we propose is implemented using a web application, sensors, and Arduino-based hardware that is connected to the database. The system helps a user know the availability of parking spaces on a real-time basis.

1.1 Problem Statement

Design a parking monitoring and control system to count the number of automobiles entering and leaving a parking space, open and close the gate automatically, provide information about free parking spaces, and create a database to provide statistics about automobiles entering and leaving the parking space.

1.2 Objective

The primary objective of this report is to design and develop a smart car parking system using a combination of HTML, PHP, CSS, and SQL technologies. This system aims to provide a seamless parking experience for users while maximizing the utilization of available parking spaces. Key functionalities of the system include real-time monitoring of parking availability, automated payment processing, and user-friendly interfaces for both administrators and customers.

1.3 Challenges and Limitations

I. Challenges

There are a number of main challenges in smart car parking projects related to processing the received data from the parking sensors including data fusion and filtering. Data fusion techniques allow to aggregate and integrate sensor data from several sources to produce knowledge. Data filtering is important to minimize as much possible the transmitted detain the network. However, where and how to conduct data fusion and filtering are still open challenges. In-network level data processing is important because the unused, unnecessary and private data is not going to be transmitted over the network. In other words, the continuously generated raw sensors data will not be sent to the cloud, which is costly and expensive. Another challenge is to identify the machine learning algorithms or capabilities that might help to process the collected data. Moreover, it is crucial to make all parties provide an excellent functionality with each other in the real time and avoid occurring of errors as much as possible.

II. Limitations

Despite its advantages, the system has several limitations. The range and accuracy of IR sensors can be affected by environmental conditions such as lighting, dirt, and obstructions, potentially leading to false readings or missed detections. The scalability of the system is also constrained by the number of available I/O pins on the Arduino, requiring additional hardware like multiplexers or additional microcontrollers for large-scale implementations. Managing power supply requirements is crucial to ensure consistent operation, particularly when scaling up the system. The integration of IoT capabilities introduces dependencies on network reliability and security. Potential issues include latency in real-time updates, vulnerability to cyber-attacks, and the need for a robust and secure web server. Additionally, the system might require significant modifications to handle complex parking scenarios, such as dynamic pricing, reservation management, or integration with existing parking management systems. As with any automated system, regular maintenance and calibration are necessary to ensure long-term accuracy and reliability.

1.4 Research Questions and Hypothesis:

I. Research Questions

1. How accurately can the IR sensors detect the presence of a vehicle in a parking space under various environmental conditions?
2. What is the optimal configuration and placement of IR sensors to minimize detection errors in a multi-space parking lot?
3. How can integrating servo motors and IR sensors improve the efficiency of parking space management compared to traditional methods?
4. What are the potential security risks associated with the web-based monitoring and control system, and how can they be mitigated?
5. How does the web-based interface affect the usability and user experience of the parking management system?
6. What are the network requirements and limitations for real-time data transmission in the IoT-enabled parking system?
7. How scalable is the Arduino-based system in managing many parking spaces, and what are the limitations?
8. What is the impact of integrating real-time data and remote-control capabilities on the operational efficiency of parking lot management?

II. Hypotheses

1. IR sensors can reliably detect the presence of a vehicle with an accuracy of over 95% under controlled environmental conditions.
2. Optimizing the placement of IR sensors will significantly reduce detection errors and improve the overall accuracy of the system.
3. The integration of servo motors and IR sensors will increase the efficiency of parking space management by at least 30% compared to manual methods.
4. Implementing robust encryption and authentication protocols will effectively mitigate most security risks associated with the web-based monitoring and control system.
5. The web-based interface will enhance user experience and usability, leading to higher user satisfaction and more efficient management.
6. Reliable real-time data transmission can be maintained with minimal latency if the network meets specific bandwidth and stability requirements.
7. The Arduino-based system is scalable to manage at least 50 parking spaces

effectively, beyond which additional hardware and software optimizations are required.

8. Integrating real-time data and remote-control capabilities will significantly improve the operational efficiency of parking lot management, reducing the time required to manage spaces and increasing overall effectiveness.

.

Chapter2

SYSTEM DESIGN

The system architecture of the smart car parking system plays a pivotal role in ensuring its reliability, scalability, and performance. It encompasses the arrangement of software components, hardware infrastructure, communication protocols, and deployment environments necessary for the system's operation.

2.1 Web Technologies

Web technology refers to how computers communicate with each other using markup languages and multimedia packages. It gives us a way to interact with hosted information, like websites. Web technology involves the use of hypertext markup language (HTML) and cascading style sheets (CSS). Many other technologies are available that help us to create websites best suited to our needs. We will learn more about these technologies in the following sub-sections.

I. HTML5

HTML5 is a markup language used for structuring and presenting content on the Worldwide Web. It is the fifth and current major version of the HTML standard. It was published in October 2014 by the World Wide Web Consortium (W3C) to improve the language with support for the latest multimedia, while keeping it both easily readable by humans and consistently understood by computers and devices such as web browsers, parsers, etc. HTML5 is intended to subsume not only HTML 4 but also XHTML 1 and DOM Level 2 HTML.HTML5 includes detailed processing models to encourage more interoperable implementations; it extends, improves, and rationalizes the markup available for documents, and introduces markup and application programming interfaces (APIs) for complex web applications. For the same reasons, HTML5 is also a candidate for cross-platform mobile applications, because it includes features designed with low-powered devices in mind. HTML documents consist of a hierarchy of elements. Each element is represented by a pair of tags, an opening tag, and a closing tag.

II. CSS

Cascading Style Sheets (CSS) is a style sheet language used for describing the presentation of a document written in a markup language like HTML. CSS is a

cornerstone technology of the World Wide Web, alongside HTML and JavaScript. One requirement for a web style sheet language was for style sheets to come from different sources on the web. Therefore, existing style sheet languages like DSSSL and FOSI were not suitable. CSS, on the other hand, lets a document's style be influenced by multiple style sheets by way of "cascading" styles.CSS is designed to enable the separation of presentation and content, including layout, colors, and fonts. This separation can improve content accessibility, provide more flexibility and control in the specification of presentation characteristics, enable multiple web pages to share formatting by specifying the relevant CSS in a separate .css file, and reduce complexity and repetition in the structural content. Separation of formatting and content also makes it feasible to present the same markup page in different styles for different rendering methods, such as on-screen, in print, by voice (via speech-based browser or screen reader), and on Braille-based tactile devices also has rules for alternate formatting if the content is accessed on a mobile device. The name Cascading Comes from the specified priority scheme to determine which style rule applies if more than one rule matches a particular element.

III. PHP

Hypertext Preprocessor (or simply PHP) is a server-side scripting language designed for Web development and also used as a general-purpose programming language. PHP code may be embedded into HTML code, or it can be used in combination with various web template systems, web content management systems, and web frameworks. PHP code is usually processed by a PHP interpreter implemented as a module in the web server or as a Common Gateway Interface (CGI) executable. The web server combines the results of the interpreted and executed PHP code, which may be any type of data, including images, with the generated web page. PHP code may also be executed with a command-line interface (CLI) and can be used to implement standalone graphical applications. The standard PHP interpreter, powered by the Zend Engine, is free software released under the PHP License. PHP has been widely ported and can be deployed on most web servers on almost every operating system and platform, free of charge. The reason behind the popularity of PHP is its several advantages. PHP is most suited for web development.

IV. Database Management System

A database management system (DBMS) is system software for creating and managing databases. The DBMS provides users and programmers with a systematic way to create, retrieve, update, and manage data. A DBMS makes it possible for end users to create, read, update, and delete data in the database. The DBMS essentially serves as an interface between the database and end users or application programs, ensuring that data is consistently organized and remains easily accessible. The DBMS manages three important things: the data, the database engine that allows data to be accessed, locked, and modified -- and the database schema, which defines the database's logical structure. These three foundational elements help provide concurrency, security, data integrity, and uniform administration procedures. Typical database administration tasks supported by the DBMS include change management, performance monitoring/tuning, and backup and recovery. Many database management systems are also responsible for automated rollbacks, restarts, and recovery as well as the logging and auditing of activity.

V. SQL

SQL (Structured Query Language) is a domain-specific language used in programming and designed for managing data held in a relational database management system (RDBMS).SQL offers two main advantages: first, it introduced the concept of accessing many records with one single command and second, it eliminates the need to specify how to reach are cord, e.g. with or without an index. Originally based upon relational algebra and tuples relational SQL consists of a data definition language, data

Fig 2.1 Web Technologies Used

manipulation language, and data control language. first, it introduced the concept of accessing many records with one single command; and second, it eliminated the need to specify how to reach are cord, e.g. with or without an index.

1. **User Interface (UI):** The front end is responsible for presenting the user interface, enabling users to interact with the system seamlessly.
   - HTML (Hypertext Markup Language): For structuring web pages. CSS (Cascading Style Sheets): For styling and layout.
   - JavaScript: For adding interactivity and dynamic behavior to the UI.
   - Bootstrap or Material-UI: CSS frameworks for creating responsive and mobile-first UI
2. **User Experience (UX):** UX design focuses on enhancing the overall user experience and usability of the system.
   - User research and personas: Understanding user needs and behaviors. Wireframing and prototyping tools: Sketch, Figma, Adobe XD.
   - Usability testing: Conducting tests to evaluate the effectiveness of the UI and UX design.
3. **Integration with Backend:** Frontend components communicate with the backend to fetch data, submit forms, and perform various actions.
   - RESTful APIs (Representational State Transfer): For communication between frontend and backend.
   - AJAX (Asynchronous JavaScript and XML) or Fetch API: For making asynchronous HTTP requests to the backend.
   - Web Sockets: For real-time communication between the client and server.
4. **Backend:** Server-side Logic: The backend handles business logic, and data processing, and interacts with databases and external services.
   - Node.js or Python: Server-side scripting languages for building scalable and efficient backend applications.
   - Express.js (Node.js) or Flask/Django (Python): Web frameworks for building RESTful APIs and handling HTTP requests.
   - Socket.IO: For implementing real-time communication between server and clients using Web Sockets.
   - **Database Management:** Backend systems store and manage data using
   - MongoDB, PostgreSQL, MySQL: Relational or NoSQL databases for

storing structured or unstructured data.

- Mongoose (for MongoDB): An object data modeling (ODM) library for Node.js that provides a schema-based solution to model application data.

2.2 System Requirement

I. User Authentication

The system should provide secure user authentication mechanisms, including login, and logout functionalities. Users should be able to create accounts with unique usernames and passwords.

II. **Booking**

Users should be able to reserve parking slots in advance. The system should allow users to select preferred dates and times for parking reservations. Confirmation emails or notifications should be sent to users upon successful booking.

III. Administrator Dashboard

Administrators should have access to a dashboard for managing parking spaces, and users. The dashboard should provide functionalities for adding, editing, and deleting parking slots. Reports should be available for monitoring system activities.

IV. User Interface

The user interface should be intuitive, responsive, and accessible across different devices and screen sizes. Clear navigation menus, buttons, and forms should facilitate easy interaction for users. Visual cues such as color coding and icons should enhance usability and user experience.

V. Data Security and Privacy

The system should implement measures to ensure the security and privacy of user data. Personal information such as names, contact details, and payment credentials should be encrypted and protected against unauthorized access. Compliance with data protection regulations such as GDPR (General Data Protection Regulation) should be ensured.

VI. Scalability and Performance

The system should be scalable to accommodate a growing number of users and parking spaces. Performance optimizations should be implemented to minimize loading times and ensure smooth operation during peak traffic periods. Load

testing should be conducted to evaluate system performance under various levels of user traffic.

VII. Entry and Exit Automation

IoT devices at entry and exit points should facilitate automated entry and exit processes for registered users. Users should be able to seamlessly enter and exit parking lots without manual Intervention.

2.3 Product Planning

This product, like any product that shall be made, needs a feasibility study to ensure that this product can be made in terms of the availability of components, cost, implementation, professional technical skills required, safety requirements, and most importantly its need by the customers. Our product's components were checked in the local market and found them available relatively not costly. However, some challenges would play roles in the progress of product design. One of them is that the team was not sure whether the team could go to the level of programming a microcontroller due to the lack of knowledge and experience in dealing with it in terms of theoretical or practical experience. Also, the controller circuit requires good skills and experience in programming and interfacing the microcontroller with the sensors, display, and database computer which might impede the product development process and success. Another challenging factor is time management since all of the team members are full of tasks other than the project which would impede as well the progress of the product development.

2.4 Customer Needs and Product Specifications

The need for such a product interests several customers. So, contacting the target customers was an essential step in studying the needs and specifications required by customers. Therefore, this was done by collecting data from the field through conducting surveys on the targeted parking to have a clear idea of what the customers need. Also, meeting or interviewing the targeted managers of the targeted location to emphasize the advantages of the product and how that product will increase their income by gaining more satisfaction from their customers. A security officer of D-mart was asked about the need for such a monitoring system and how it would be beneficial to them. He said that it would be very beneficial since in the morning 3 security officers are working in the parking area trying just to guide people and close the entrances of those filled parking spaces. They highly appreciate such a system to be

implemented in D-mart. Also, ordinary people were asked about parking lots in malls and airports, and similar responses we received. Another step is to analyze such data to know how it can be implemented cost-effectively and efficiently in line with the features preferred by customers. Then, the gathered information was analyzed to define the product specifications that match needs and requirements taking into consideration any safety issues or standards. According to customer needs, all they have in common is to find free parking spaces easily without detailed and specific technology to be used. However, some prefer a display to show the number of free parking spaces in addition to an indication of whether a particular zone is full or not.

2.5 Concept Selection

Selecting the best alternative among several options is mainly dependent on the availability of components, cost, reliability, implementation, professional technical skills required, safety requirements, and most importantly its need by the customers. Therefore, studying the alternatives, in terms of all essential design steps to select the best one, requires the right decision-making. Then testing the selected option against several factors such as customer preferences and needs, national standards if it shall be recommended by government law, availability of material, estimated material costs, financial budget, feasibility as well and efficiency of which all will determine the final selection of the design. Selecting the best alternative among several options is mainly dependent on the availability of components, cost, reliability, implementation, professional technical skills required, safety requirements, and most importantly its need by the customers. Therefore, studying the alternatives, in terms of all essential design steps to select the best one, requires the right decision-making, then testing the selected option against several factors such as customer preferences and needs, national standards if it shall be recommended by government law, availability of material, estimated material costs, financial budget, feasibility as well as efficiency of which all will determine the final selection of the design. Selecting the best alternative among several options is mainly dependent on the availability of components, cost, reliability, implementation, professional technical skills required, safety requirements, and most importantly its need by the customers.

2.6 Use Case:

A use case diagram is a visual representation of the interactions between users, known as actors, and a system to accomplish specific tasks or goals. It provides a high-level view of the system's functionality from the perspective of its users. Actors are external entities interacting with the system, which can include human users, other systems, or even hardware devices. Use cases represent the various actions or functionalities offered by the system to fulfill the needs of its actors. These functionalities are depicted as ovals within the diagram, with lines connecting them to the respective actors to illustrate the interactions. Additionally, use case diagrams may include relationships such as "include" and "extend" to capture common or optional behaviors shared among multiple use cases. Use case diagrams serve as valuable tools.

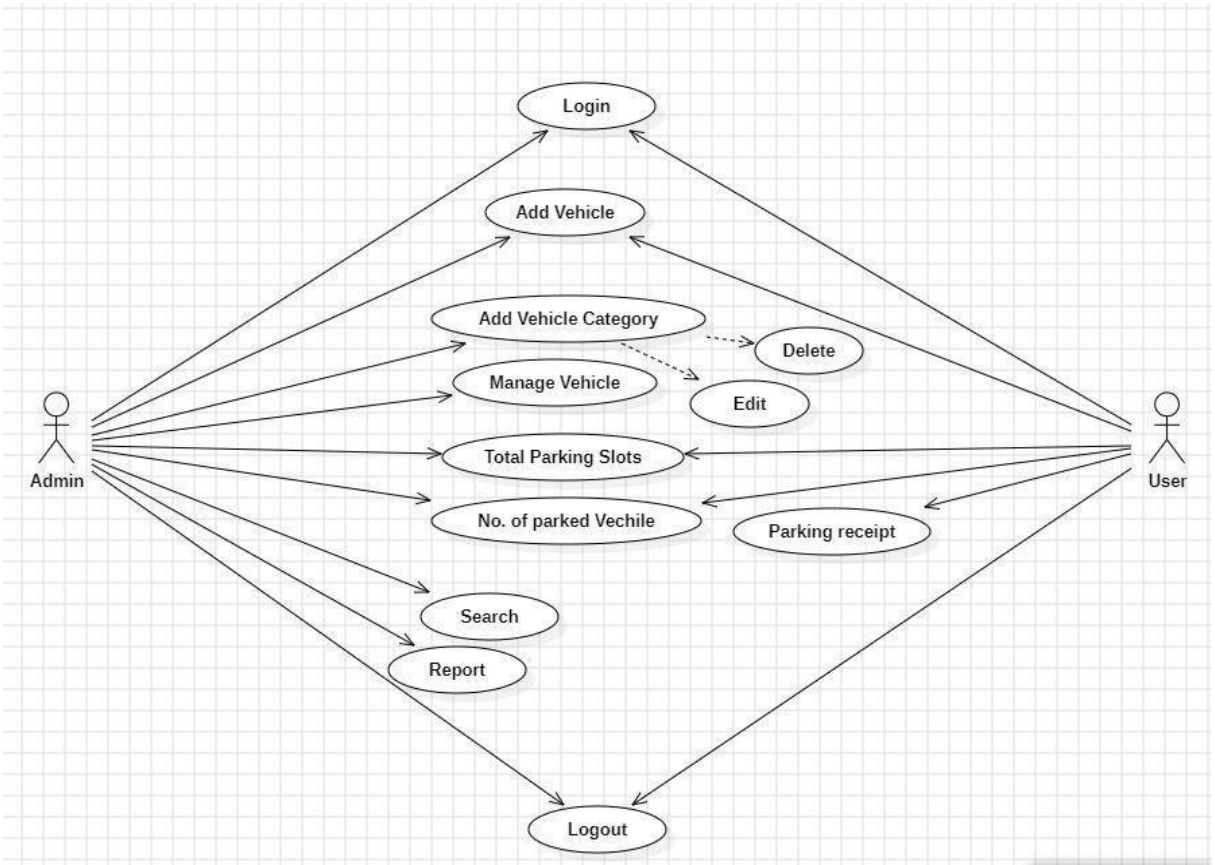


Fig 2.2 Use case diagram for Smart Parking

in requirement analysis, system design, and communication among stakeholders during the software development process. They help clarify the system's scope, define user requirements, and facilitate discussions about system functionality and behavior.

Actor: - Admin	**Actor: - User**
Log In	Login
Search Parking Spaces Reserve	Add vehicle
Parking Space Add vehicle	Total parking slots No. of parked
Add vehicle category	vehicle Parking Receipt
Manage vehicle	Logout

Relationships

Users interact with the system to perform various tasks like registration, login, searching for parking spaces, monitoring availability, and automated entry/exit. The system interacts with external IoT devices to update parking availability.

In this IoT parking system, administrators and users interact with distinct sets of functionalities tailored to their roles. Administrators, represented as the admin actor, possess administrative privileges and responsibilities. They are tasked with system management, including user and vehicle management, space reservation, and oversight of parking operations. Admins can log into the system to access administrative tools such as searching for available parking spaces, reserving spaces, adding new vehicles, defining vehicle categories, managing existing vehicle records, and generating reports on various aspects of parking operations like occupancy rates and revenue. Additionally, admins can monitor the total number of parking slots available and the number of vehicles currently parked within the facility, enabling them to efficiently manage parking resources and address any issues that arise.

On the other hand, users, represented as the User actor, engage with functionalities designed to facilitate their parking experience. Users log into the system to access features such as adding their vehicles to the system, checking the total number of available parking slots, viewing the current count of parked vehicles, generating parking receipts, and securely logging out. By registering their vehicles and accessing real-time information on parking availability, users can streamline their parking experience, optimize their search for parking spaces, and ensure a smooth transition into and out of the parking facility. This segregation of functionalities between administrators and users ensures that each stakeholder group can effectively fulfill their respective roles within the IoT parking system, contributing to the overall efficiency and user satisfaction

Chapter 3

SYSTEM HARDWARE

The hardware components of an Arduino-based car parking system typically include a servo motor, infrared (IR) sensors, and an Arduino microcontroller. The servo motor is responsible for moving barriers or indicators to show parking availability. It operates by receiving PWM (pulse width modulation) signals from the Arduino to precisely control its position. IR sensors are used to detect the presence of a vehicle in a parking spot. These sensors emit infrared light and measure the reflection to determine if an object is present, providing real-time data to the Arduino. The Arduino microcontroller acts as the brain of the system, processing inputs from the IR sensors and controlling the servo motor based on the sensor data. This integration of components allows for efficient and automated management of parking spaces.

3.1 Arduino Uno Board

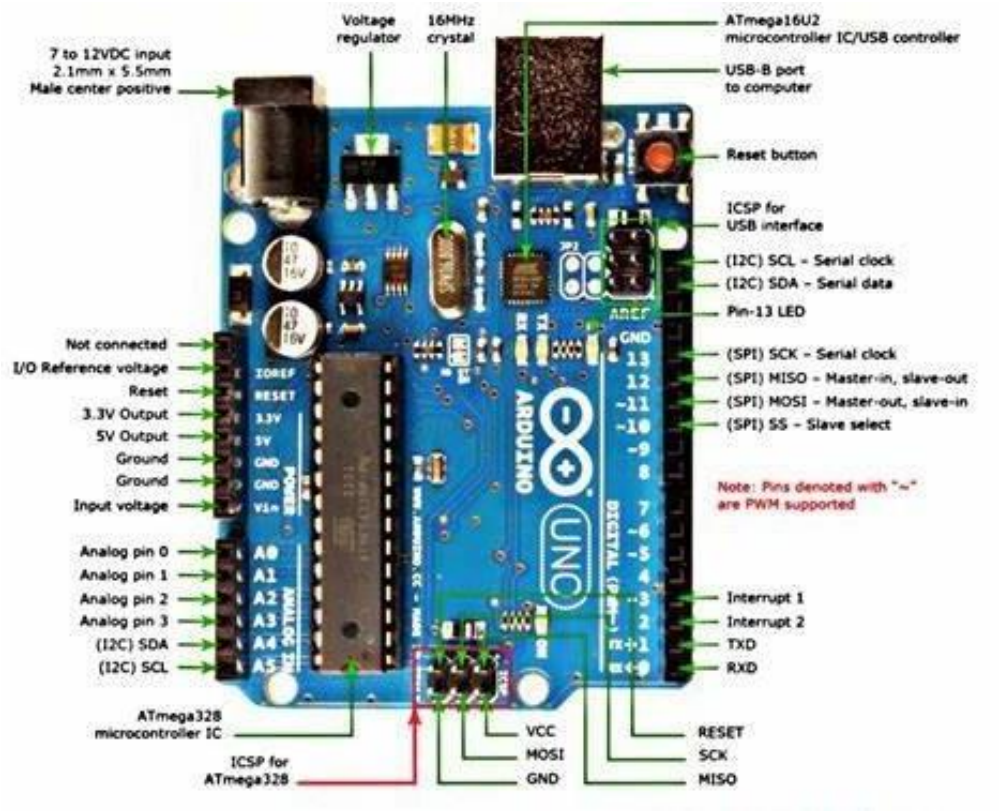


Fig 3.1 Arduino Uno Board

The Arduino Uno is an open-source microcontroller board based on the Microchip ATmega328P microcontroller and developed by Arduino. The board is equipped with sets of digital and analog input/output (I/O) pins that may be interfaced to various expansion boards (shields) and other circuits. The board has 14 digital I/O pins (six capable of PWM output), 6 analog I/O pins, and is programmable with the Arduino IDE (Integrated Development Environment), via a type B USB cable. It can be powered by the USB cable or by an external 9-volt battery, though it accepts voltages between 7 and 20 volts. It is similar to the Arduino Nano and Leonardo. The hardware reference design is distributed under a Creative Commons Attribution Share-Alike 2.5 license and is available on the Arduino website. Layout and production files for some versions of the hardware are also available. The Arduino Uno is a cornerstone in the world of microcontrollers, revered for its accessibility, versatility, and ease of use. At its heart lies the ATmega328P microcontroller, an 8-bit AVR chip clocked at 16 MHz, providing a robust platform for a myriad of projects. What sets the Uno apart is its array of digital and analog input/output pins. With 14 digital pins, 6 of which can double as PWM outputs, and 6 analog input pins, the Uno offers ample opportunities for interfacing with sensors, actuators, and other devices.

3.2 LCD Display

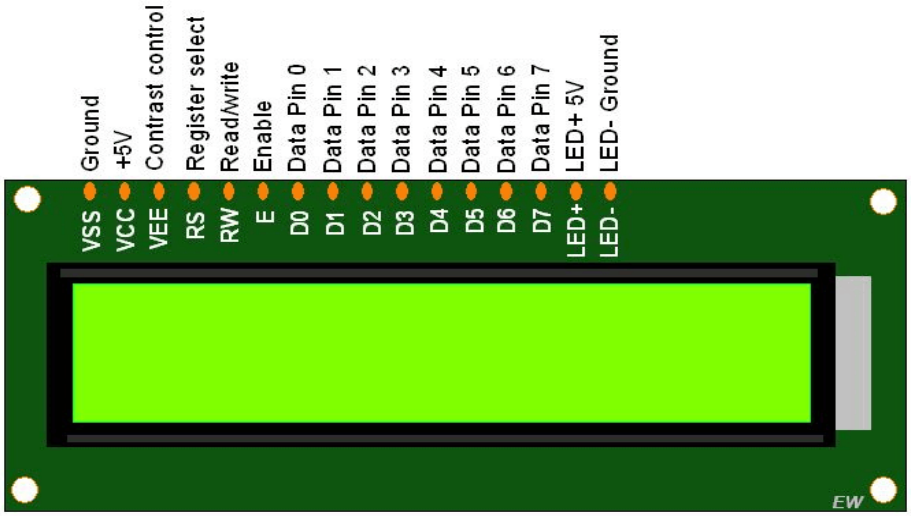


Fig 3.2 LCD Display

A liquid-crystal display (LCD) is a flat-panel display or other electronically modulated optical device that uses the light-modulating properties of liquid crystals combined with polarizers. Liquid crystals do not emit light directly, instead using a backlight or reflector to produce images in color or monochrome. Lcds are available to display

arbitrary images (as in a general-purpose computer display) or fixed images with low information content, which can be displayed or hidden. For instance: preset words, digits, and seven segment displays, as in a digital clock, are all good examples of devices with these displays. They use the same basic technology, except that arbitrary images are made from a matrix of small pixels, while other displays have larger elements. Lcds can either be normally on (positive) or off (negative), depending on the polarizer arrangement. For example, a character positive LCD with a backlight will have black lettering on a background that is the color of the backlight, and a character negative LCD will have a black background with the letters being of the same color as the backlight. Optical filters are added to white on blue lcds to give them their characteristics appearance. Liquid crystals do not emit light directly,[1] instead using a backlight or reflector to produce images in color or monochrome. Lcds are available to display arbitrary images (as in a general-purpose computer display) or fixed images with low information content, which can be displayed or hidden. . For instance: preset words, digits, and seven segment displays, **as** in a digital clock, are all good examples of devices with these displays.

3.3 I2C Converter

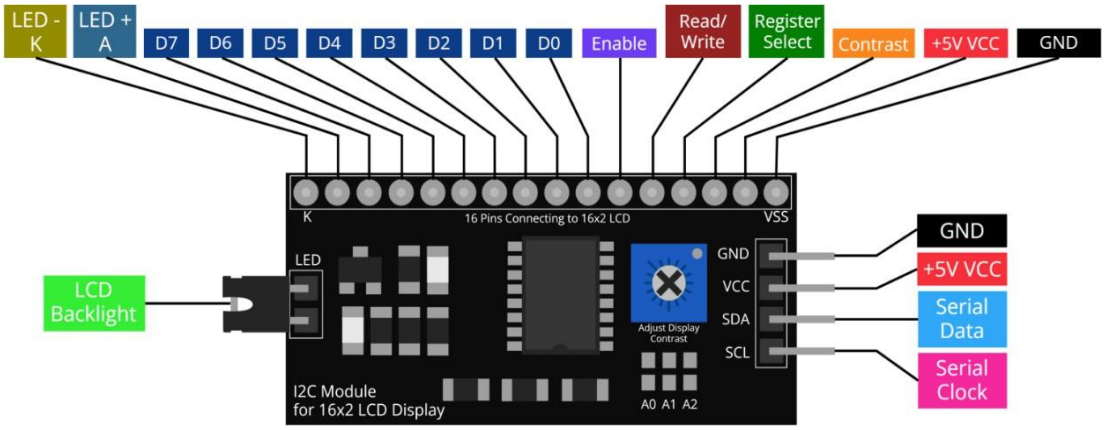


Fig 3.3 I2C Converter

I2C (Inter-Integrated Circuit, eye-squared-C), alternatively known as I2C or IIC, is a synchronous, multi-controller/multi-target (controller/target), packet switched, single ended, serial communication bus invented in 1982 by Philips Semiconductors. It is widely used for attaching lower speed peripheral ICs to processors and microcontrollers in short distance, intra-board communication. Several competitors, such as Siemens, NEC, Texas Instruments, STMicroelectronics, Motorola, Nordic Semiconductor and

Intersil, have introduced compatible I2C products to the market since the mid1990s. System Management Bus (SMBus), defined by Intel in 1995, is a subset of I 2C, defining a stricter usage. One purpose of SMBus is to promote robustness and interoperability. Accordingly, modern I2C systems incorporate some policies and rules from SMBus sometimes supporting both I2C and SMBus, requiring only minimal reconfiguration either by commanding or output pin use. The converter typically operates by translating the data and control signals between the two protocols, effectively acting as a protocol translator. It may incorporate level shifting circuitry to ensure compatibility between devices operating at different voltage levels. Additionally, some I2C converters may offer features such as signal buffering, clock synchronization, or configurable addressing to optimize communication between devices. In summary, the I2C converter plays a crucial role in integrating diverse devices into I2C-based systems, enabling seamless communication and interoperability across different communication protocols. Its versatility and flexibility make it an essential component in embedded systems design, facilitating the development of complex and interconnected electronic systems

3.4 Servo Motor

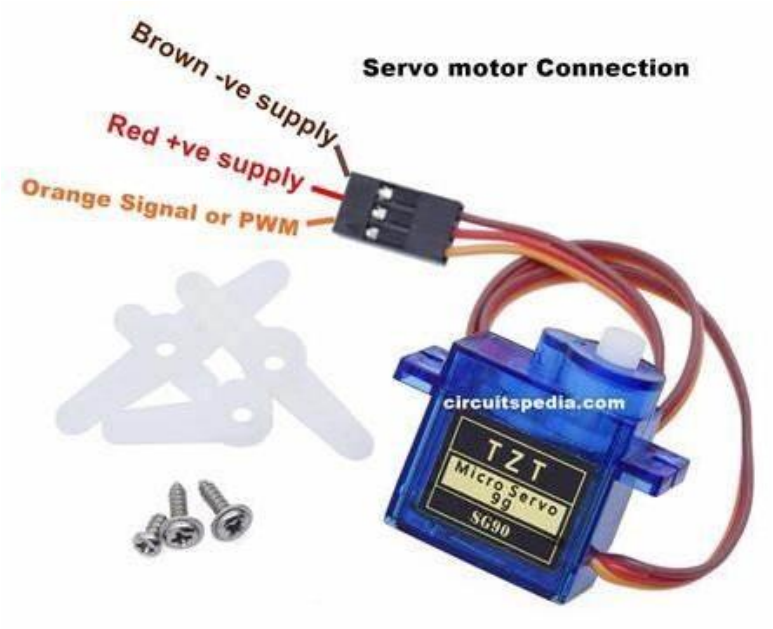


Fig 3.4 Servomotor

A servomotor is designed to move to a given angular position. A typical servo motor has three connections. Two of them are the positive and 0V supply lines. The third connection carries the control signal pulses from the control circuit (the Arduino in this case). Servo motors may be classified, according to the torque it can withstand, as mini, standard and giant servos. Usually mini and standard size servo motors can be controlled by Arduino directly with no need to external driver. The rotor of the motor has limited ability to turn. Generally, it can turn 60-90° on either side of its central position. The control signal is a series of pulses transmitted at intervals of about 18 ms, or 50 pulses per second. The angle (of mechanical rotation) is determined by the width of an electrical pulse that is applied to the control wire. This is a form of pulse-width modulation; however, servo position is not defined by the PWM duty cycle (i.e., ON vs. OFF time) but only by the width of the pulse. The width of the pulse will determine how far the motor turns. For example, a 1.5 ms pulse will make the motor turn to the 90-degree position (neutral position) as shown in Figure 5 ms turn as far as possible to the left. A2 ms turn as far as possible to the right. Intermediate pulse lengths give intermediate positions. Servo motors are typically characterized by their torque, speed, and resolution. Torque refers to the rotational force produced by the motor, while speed indicates how quickly the motor can rotate. Resolution, on the other hand, refers to the smallest increment of motion that the motor can achieve.

3.5 IR Sensor

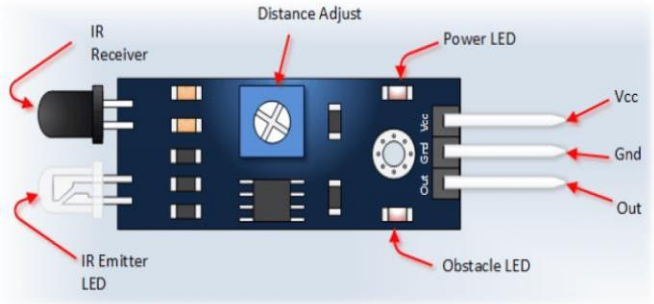


fig 3.5 IR Sensor

An infrared sensor (PIR sensor) is an electronic sensor that measures infrared (IR) light radiating from objects in its field of view. They are most often used in PIR-based motion detectors. PIR sensors are commonly used in security alarms and automatic lighting applications. PIR sensors detect general movement, but do not give information on who or what moved. For that purpose, an imaging IR sensor is required. PIR sensors are

commonly called simply "PIR", or sometimes "PID", for "passive infrared detector". The term passive refers to the fact that PIR devices do not radiate energy for detection purposes. They work entirely by detecting infrared radiation (radiant heat) emitted by or reflected from objects.

3.6 Breadboard

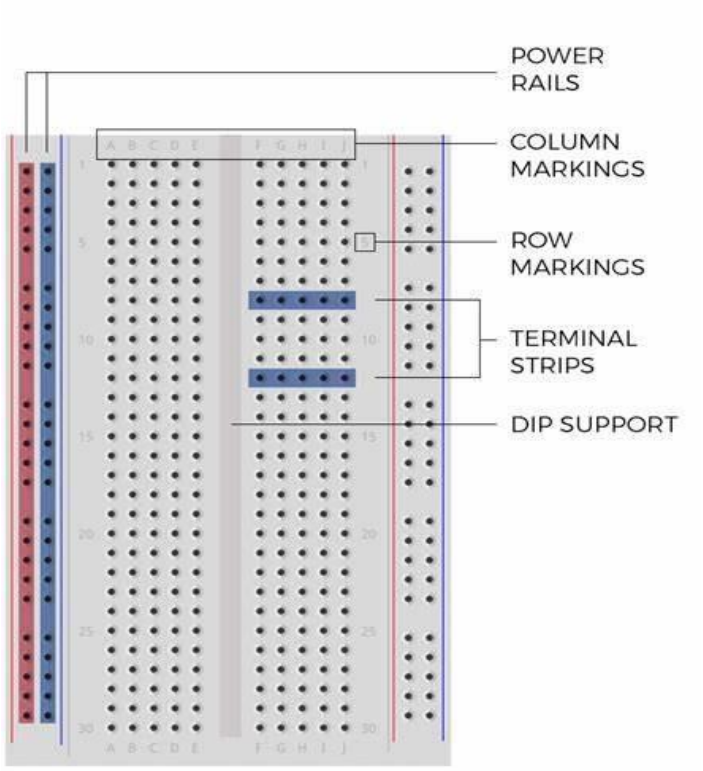


Fig 3.6 Breadboard

A breadboard is a fundamental tool in prototyping electronic circuits, including those used in IoT (Internet of Things) projects. It consists of a plastic board with a grid of holes into which electronic components can be inserted and interconnected without the need for soldering. This allows for rapid and flexible experimentation with various sensors, actuators, microcontrollers, and other IoT components. Breadboards typically have rows and columns of interconnected metal strips beneath the surface, enabling easy connection of components using jumper wires. They're indispensable for testing and iterating on IoT designs, as components can be easily swapped and rearranged to explore different configurations. Additionally, breadboards often come in various sizes and configurations to accommodate projects of different complexities and scales, making them a versatile tool for IoT development

3.7 Jumper Wires

Fig 3.7 Jumper Wires

Jumper wires are essential components in electronics prototyping, facilitating the connection of various elements within circuits on breadboards or other platforms. These wires are typically made of flexible, insulated material with metal tips that can be easily inserted into the holes of a breadboard, connecting different points to establish electrical pathways. Jumper wires come in different lengths, colors, and configurations, allowing for precise and organized wiring arrangements. They enable rapid iteration and experimentation in electronics projects by providing a quick and temporary means of establishing connections between components. Whether bridging gaps between components, creating branches in circuits, or troubleshooting connections, jumper wires offer flexibility and convenience in prototyping and testing electronic designs. Their simplicity and versatility make them indispensable tools for hobbyists, students, and professionals alike in exploring and developing various electronic systems and devices.

3.8 Cable Wires

Fig 3.8 Cable wires

Cable wires for power supply play a critical role in transmitting electrical power from a source to electronic devices, equipment, or systems. These wires are typically insulated conductors designed to carry electrical current safely and efficiently. Power supply cables come in various types and configurations depending on the specific requirements of the application, such as voltage, current capacity, and environmental conditions. They may include AC (alternating current) cables for mains power or DC (direct current) cables for battery-powered devices. Additionally, power supply cables often feature different connectors, such as USB, barrel connectors, or proprietary plugs, to ensure compatibility with the corresponding power sources and devices. The insulation material and construction of power supply cables are chosen to withstand the voltage levels and environmental factors encountered during operation, ensuring reliable and safe power transmission. Overall, cable wires for power supply are essential components in electrical systems, providing the necessary connection between power sources and devices to enable their proper function.

3.9 Arduino IDE Software

The Arduino Integrated Development Environment (IDE) is a software application that provides a comprehensive platform for programming Arduino microcontroller boards. It offers a user-friendly interface that simplifies the process of writing, compiling, and uploading code to Arduino boards. The IDE is compatible with Windows, macOS, and Linux operating systems, making it accessible to a wide range of users. It features a text editor with syntax highlighting and code auto-completion, which aids in writing and editing Arduino sketches (programs). Additionally, the IDE includes a built-in compiler that translates the written code into machine-readable instructions for the Arduino board. Users can easily verify their code for errors and compile it into a binary file ready for uploading. The IDE also facilitates the uploading process by providing a straightforward interface for selecting the target Arduino board and communication port. Moreover, the Arduino IDE supports a vast library of pre-written code examples and libraries, enabling users to quickly incorporate functionality such as sensor readings, communication protocols, and device control into their projects. Overall, the Arduino IDE serves as a central hub for Arduino development, offering a seamless environment for both beginners and experienced users to create and deploy projects with Arduino microcontrollers.

Fig 3.9 Arduino IDE Software

1. Debugger

The Arduino IDE does not have a built-in debugger. However, developers can use serial output for debugging purposes. They can use functions like Serial. Print () to output variable values or debug messages to the serial monitor for analysis during runtime.

2. Sketchbook

The Sketchbook is a folder on your computer where Arduino sketches (programs) are stored. In the Arduino IDE, you can access your Sketchbook through the "File" menu. It's where you can organize, create, and open your sketches. By default, the Sketchbook folder is located in the Arduino directory in your documents folder.

3. Library Manager

The Library Manager is a feature of the Arduino IDE that allows users to easily discover, install, and manage libraries (collections of pre-written code) for their projects. Users can access the Library Manager through the "Sketch" menu > "Include Library" > "Manage Libraries...". From there, they can search for libraries by name, install new libraries, update existing ones, and remove libraries they no longer need.

4. Board Manager

The Board Manager is a tool within the Arduino IDE that enables users to add support for different Arduino-compatible boards. Users can access the Board Manager through

the "Tools" menu > "Board" > "Board Manager...". From there, they can search for and install board support packages (BSPs) for various Arduino boards, including official Arduino boards as well as third-party boards from different manufacturers.

5.Tools:

The "Tools" menu in the Arduino IDE contains various tools and settings that users can configure for their projects. This includes options for selecting the Arduino board, setting the processor frequency, choosing the programmer, and configuring serial port settings for uploading sketches.

Chapter 4

METHODOLOGY

The methodology for implementing a car parking management system using IoT involves a systematic approach to design, develop, and deploy a solution that efficiently monitors and manages parking spaces. Initially, the project's objectives and scope are defined, outlining the goals and functionalities expected from the system. Following this, a thorough review of existing literature and technologies related to IoT-based parking management systems is conducted to gather insights and inform the design process. Requirement analysis involves identifying stakeholders' needs, including parking facility managers and users, to determine functional and nonfunctional requirements.

The system design phase focuses on architecting the overall solution, including hardware components such as Arduino microcontrollers and sensors, as well as software components like communication protocols and data processing algorithms. Hardware implementation involves procuring and assembling the necessary components, while software development entails writing code for Arduino devices to collect and transmit parking occupancy data. Integration and testing are crucial stages where the hardware and software components are combined, and the system's functionality and performance are evaluated. Deployment involves installing the IoT-based parking management system in a real-world environment, followed by ongoing maintenance and support to ensure its continued operation. Throughout the methodology, documentation and knowledge sharing are emphasized to capture project findings and disseminate insights to stakeholders and the broader community.

4.1 Process Model

Iterative process starts with a simple implementation of a subset of the software requirements and iteratively enhances the evolving versions until the full system is implemented. At each iteration, design modifications are made and new functional capabilities are added. Iterative and Incremental development is any combination of both iterative design or iterative method and incremental build model for software development. The combination is of long standing and has been widely suggested for large development efforts. For example, the 1985 DOD-STD-2167 mentions During software development,

more than one iteration of the software development cycle may be in progress at the same time. and "This process may be described as an 'evolutionary acquisition' or 'incremental build' approach." The relationship between iterations and increments is determined by the overall software development methodology and software development process. The exact number and nature of the particular incremental builds and what is iterated will be specific to each individual development effort. An iterative life cycle model does not attempt to start with a full specification of requirements. Instead, development begins by specifying and implementing just part of the software, which can then be reviewed in order to identify further requirements. Parking Management System process is then repeated, producing a new version of the software for each cycle of the model.

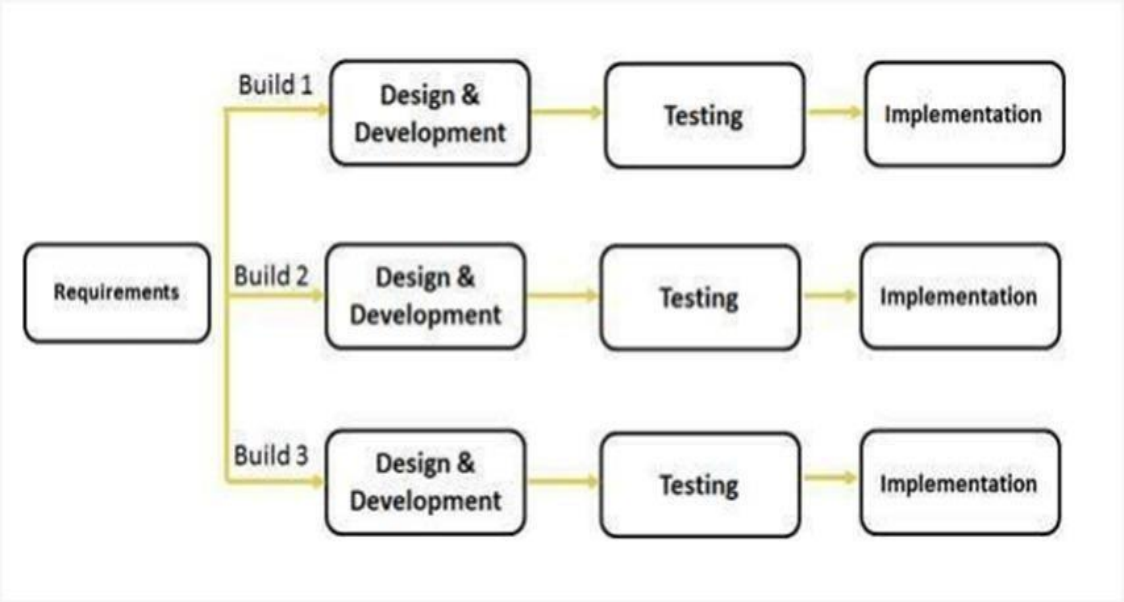


Fig 4.1 Iterative Model

The iterative model is a software development methodology characterized by its cyclic approach to project execution, where design, development, implementation, and testing activities are repeated in successive iterations. Initially, the project's requirements are gathered and analyzed, informing the design phase. During this phase, the system architecture, user interface, and other design elements are conceptualized. Once the design is established, development begins, where the system's functionality is implemented according to the design specifications. This phase involves writing code, integrating components, and building the system incrementally. As development progresses, each iteration results in a functional prototype or increment of the system.

Implementation follows the development phase, where the prototype is deployed in a real-world environment or test environment. This allows stakeholders to interact with the system and provide feedback for further refinement. Testing is conducted throughout the entire development lifecycle, with each iteration subject to rigorous testing to identify defects and ensure quality. Feedback from testing is used to refine the system's design and functionality, leading to subsequent iterations. The iterative model promotes flexibility and adaptability, allowing for incremental improvements and the incorporation of stakeholder feedback throughout the development process. By iterating through design, development, implementation, and testing cycles, the iterative model facilitates the creation of robust, high-quality software solutions that meet evolving requirements and user needs.

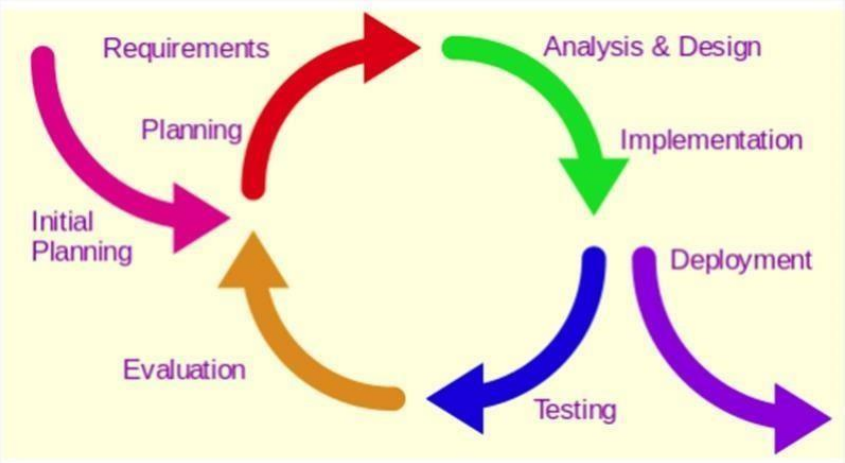


Fig 4.2 Methodology

The methodology for implementing a car parking management system using IoT involves a systematic approach to design, develop, and deploy a solution that efficiently monitors and manages parking spaces. Initially, the project's objectives and scope are defined, outlining the goals and functionalities expected from the system. Following this, a thorough review of existing literature and technologies related to IoT- based parking management systems is conducted to gather insights and inform the design process. Requirement analysis involves identifying stakeholders' needs, including parking facility managers and users, to determine functional and non- functional requirements. The system design phase focuses on architecting the overall solution, including hardware components such as Arduino microcontrollers and sensors, as well as software components like communication protocols and data processing algorithms. Hardware implementation involves procuring and assembling the necessary

components, while software development entails writing code for Arduino devices to collect and transmit parking occupancy data. Integration and testing are crucial stages where the hardware and software components are combined, and the system's functionality and performance are evaluated. Deployment involves installing the IoT-based parking management system in a real-world environment, followed by ongoing maintenance and support to ensure its continued operation. Throughout the methodology, documentation and knowledge sharing are emphasized to capture project findings and disseminate insights to stakeholders and the broader community.

4.2 Feasibility Study

I. Economic feasibility

Economic feasibility attempts to weigh the cost of developing and implementing a new system, against the benefits that would accurate from having the new system in place. This feasibility study gives the top management the economic justification for the new system A simple economic analysis which gives the actual comparison of costs and benefits are much more meaningful in this case. In addition, this proves to be a useful point of reference to compare actual costs as the project progresses. There could be various types of intangible benefits of account of automation. These could include increased customer satisfaction, improved accuracy of operation, better documentation and record keeping, faster retrieval of information. Schedule Feasibility means that the project can be completed on time the project does not have a deadline but according to the proposed system the development process is on schedule. Therefore, it is feasible.

II. Operational feasibility

Proposed project is beneficial only if it can be turned into information systems that will meet the organization operating requirements. Simply stated, this test of feasibility asks if the system will work when it is developed and installed. What are major barriers to implementation? Here are questions that will help test the d operational feasibility of a project.

III. Technical feasibility

Technical feasibility centers on the existing computer system (hardware, software, etc.) and to what extent it can support the proposed addition. For example, if the current computer is operating a 80% capacity-an arbitrary

ceiling-then running another application could overload the system or require additional hardware. This involves financial considerations to accommodate technical enhancements. If the budget is a serious constraint, then the project is judged but not feasible.

4.3 Implementation Phase:

I. Cost Benefit Analysis

Cost benefit analysis (CBA) estimates and total up the equivalent money value of the benefits and the cost invested to for implementation the software. Cost benefit analysis (CBA) is the weighing scale approach to decision-making. All the plus points (such as cash flow and other intangible benefits) are put on one side all the minus points (the cost and disadvantages) are put on the other side. Both sides should be weighed and benefits should be evaluated.

II. Cost Estimation

A cost estimate is the approximation of the cost of a program, project, or operation. The cost estimate is the product of the cost estimating process. The cost estimate has a single total value and may have identifiable component values. For a given set of requirements, it is desirable to know how much it will cost to develop the software to satisfy a given requirement, and how much time development will take. The cost of a project is a function of many parameters. Foremost among them is the size of the project. Other factors that affect the cost are programmer ability.

III. Benefits

Improves business processes leading to annual cost decrease. Due to availability of information, better decision making is possible leading to additional Cash flows.

4.4 Literature Review

I. Literature Search:

To initiate the literature, search for IoT-based parking management systems, it's crucial to leverage academic databases like IEEE Xplore, ACM Digital Library, and Science Direct, which offer access to a vast array of peer-reviewed research papers and articles. Additionally, utilizing search engines such as Google Scholar can broaden the scope of the search and provide access to a diverse range of sources. Using specific keywords such as "IoT-based parking

management," "smart parking systems," and "wireless sensor networks in parking" can help narrow down the search results and target relevant literature. These keywords are tailored to capture papers and articles specifically related to IoT applications in parking management, including the use of wireless sensor networks for vehicle detection and occupancy monitoring. By employing a combination of academic databases and targeted keywords, researchers can efficiently identify relevant literature to inform their study and gain insights into the current state of the field.

II. Selection Criteria

When selecting papers and articles for the literature review, it is essential to apply specific criteria to ensure the relevance and quality of the literature. One crucial criterion is the recency of the publication, with a preference for papers and articles published within the last 5-10 years. This ensures that the literature is up-to-date and reflects the latest advancements, trends, and practices in the field of IoT-based parking management systems. Additionally, prioritizing peer-reviewed journal papers, conference proceedings, and reputable sources is paramount to ensure the credibility and rigor of the literature. Peer reviewed publications undergo rigorous evaluation by experts in the field, ensuring the accuracy and validity of the research findings. Conference proceedings often feature cutting-edge research presented at academic conferences, providing valuable insights into emerging technologies and methodologies. Reputable sources, such as well-established journals and conferences, are known for maintaining high standards of scholarship and integrity. By adhering to these selection criteria, researchers can ensure that the literature review encompasses high-quality, relevant literature that contributes meaningfully to the study of IoT-based parking management systems.

III. Analysis and Synthesis

In the analysis and synthesis phase of the literature review for IoT-based parking management systems, researchers meticulously examine and consolidate the gathered information to derive meaningful insights and conclusions. This process involves identifying common patterns, trends, and best practices across the reviewed literature while also scrutinizing differences and nuances among various studies. Researchers compare and contrast the features, technologies, methodologies, and findings of different IoT-base parking

managementzsystems, aiming to distill key insights and lessons learned. By synthesizing the diverse range of information gleaned from the literature, researchers can gain a comprehensive understanding of the state of the art in IoT-based parking management. They identify strengths and weaknesses in existing approaches uncover emerging trends and challenges, and highlight opportunities for further research and development. The analysis and synthesis phase serves as the foundation for generating new knowledge and informing the design and implementation of innovative solutions in the field of smart parking. Ultimately, this process enables researchers to contribute valuable insights to the advancement of IoT technologies and their application in optimizing parking management systems.

IV. Documentation and Reporting:

In the documentation and reporting phase of the literature review for IoT-based parking management systems, researchers meticulously compile and organize their findings to produce a comprehensive and informative report. This process involves summarizing each reviewed paper or article, capturing key insights, findings, and relevant quotes or excerpts from the literature. By systematically documenting the information gathered, researchers ensure a thorough understanding of the state of the art in IoT-based parking management. Additionally, researchers organize the information into categories or themes, such as architecture, sensors, communication, and analytics, to facilitate easier analysis and reference. This categorization enables researchers to identify common patterns, trends, and best practices across the reviewed literature, providing a structured framework for synthesizing their findings. Finally, researchers prepare a comprehensive report or literature review paper that summarizes the state of the art in IoT-based parking management systems. This report highlights key findings, challenges, and opportunities identified through the literature review, offering valuable insights to inform future research and development efforts in the field. By documenting and reporting their findings in a systematic and organized manner, researchers contribute to the advancement of knowledge and the ongoing evolution of IoT technologies in parking management.

Chapter 5

IMPLEMENTATION AND WORKING

Before this section, we have discussed about the architecture and technical terminologies that are used in the smart parking. Here we are going to see how actually in real world this Smart Parking is going to work. For this, we have designed a flow chart elaborating the flow of working of Real Time Smart Car Parking System using IOT.

5.1 Block Diagram

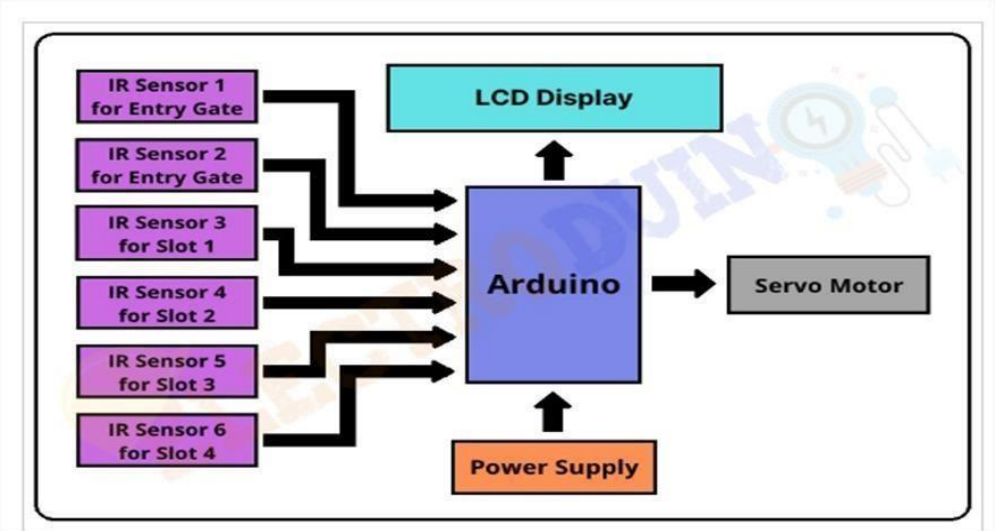


Fig 5.1 Block Diagram of Smart Parking System Project

This smart parking system project consists of Arduino, six IR sensors, one servo motor, and one LCD display. Where the Arduino is the main microcontroller that controls the whole system. Two IR sensors are used at the entry and exit gates to detect vehicle entry and exit in the parking area. And other four IR sensors are used to detect the parking slot availability. The servo motor is placed at the entry and exit gate that is used to open and close the gates. Also, an LCD display is placed at the entrance, which is used to show the availability of parking slots in the parking area. When a vehicle arrives at the gate of the parking area, the display continuously shows the number of empty slots. If there have any empty slots then the system opens the entry gate by the servo motor. After entering the car into the parking area, when it will occupy a slot, then the display shows this slot is full. If there is no empty parking slot then the system displays all slots are full and do not open the gate.

5.2 Circuit Diagram Using Arduino

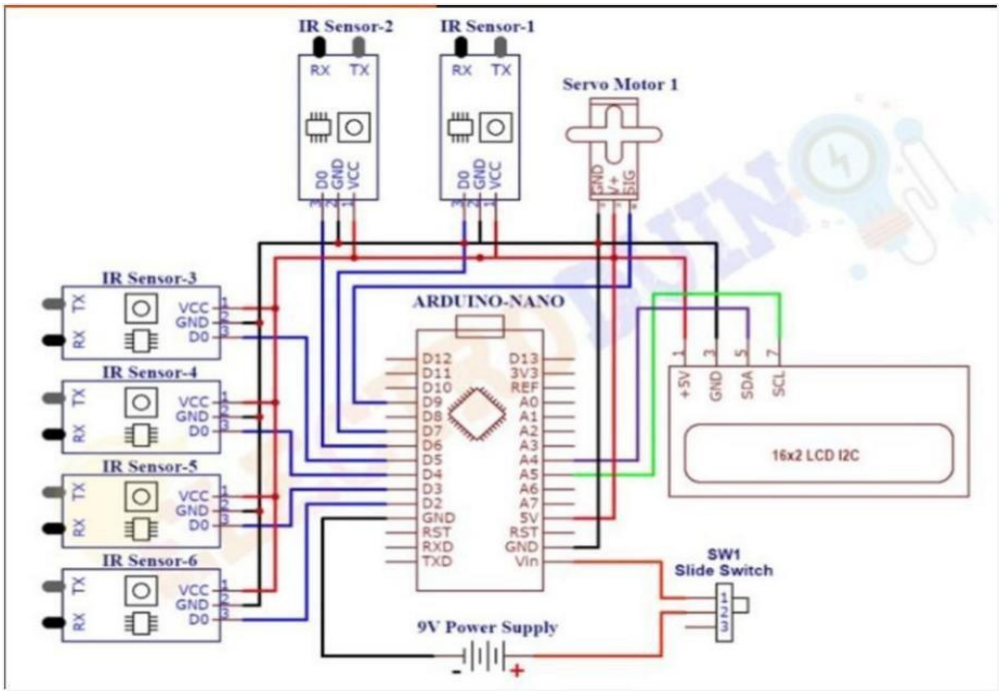


Fig 5.2 Circuit Diagram

After assembling all components according to the circuit diagram and uploading the code to the Arduino board. Now place the sensors and servo motor at accurate positions There are four parking slots in this project, IR sensor-3, 4, 5, and 6 are placed at slot-1, 2, 3, and 4 respectively. IR sensor-1 and 2 are placed at the entry and exit gate respectively and a servo motor is used to operate the common single entry and exit gate. The LCD display is placed near the entry gate. The system used IR sensor-3, 4, 5, and 6 to detect whether the parking slot is empty or not and IR sensor-1, and 2 for detecting vehicles arriving or not at the gate. In the beginning, when all parking slots are empty, then the LCD display shows all slots are empty. When a vehicle arrives at the gate of the parking area then the IR sensor1 detects the vehicle and the system allowed to enter that vehicle by opening the servo barrier. After entering into the parking area when that vehicle occupies a slot then the LED display shows that the slot is full. In this way, this system automatically allows 4 vehicles. In case the parking is full, the system blocked the entrance gate by closing the servo barrier. And the LED display shows that slot-1, 2, 3, and 4 all are full. When a vehicle leaves a slot and arrives at the gate of the parking area then the IR sensor-2 detects that vehicle and the system open the servo barrier. Then the LED display shows that the slot is empty. Again, the system will allow entering a new vehicle.

5.3 Flowchart:

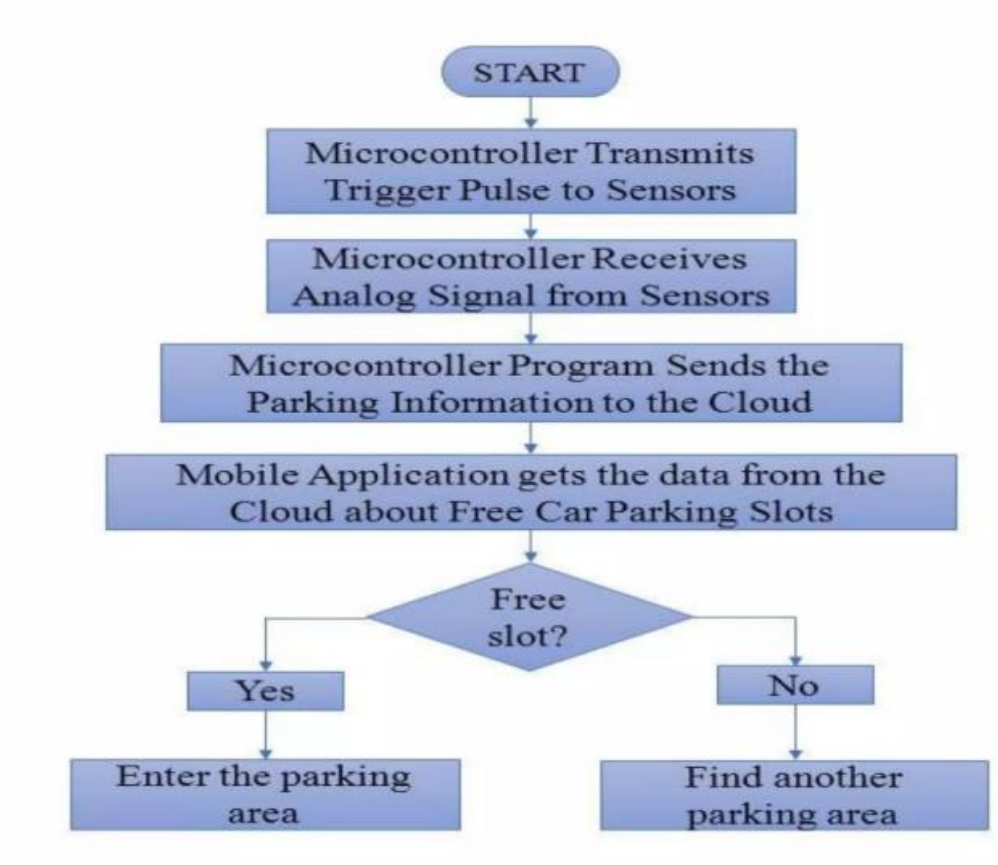


Fig 5.3 Flowchart

We have formed a real-world model representing the minimized view of the parking lot structure. We applied all the systems their and checked how actual system is going to perform.

Here are steps:

Step 1: Complete the installation of smart parking android application at user side. Step 2: Do registration and login.

Step 3: Click on book slot. You will get status of all available and parked slots. Step 4: Click on available slots to request for parking.

Step 5: Now request will be get updated to admin panel. When you reached the parking area admin will accept your request and allow you to enter.

Step 6: IR sensors are continuously detecting that if car is arrived or present or not. Step 7: If car gets detected then it gets updated on application as well as website via cloud.

Step 8: Whenever you will leave the parking lot, the particular parking slot gets free and updated on each platform.

Data collection and processing are crucial components of an IOT parking system as they involve gathering information from various sensors deployed in parking spaces and then analyzing this data to derive meaningful insights.

I. Data collection

Sensor Data IoT parking systems typically employ various types of sensors such as ultrasonic sensors, infrared sensors, or cameras to detect the presence of vehicles in parking spaces. Occupancy Status: Sensors continuously collect data on whether a parking space is occupied or vacant. Some systems may also gather additional data such as vehicle size, license plate recognition, or vehicle type for better management and analytics. Timestamps Data collection includes timestamps to record when vehicles enter or leave parking spaces, enabling tracking of parking duration.

II. Data Transmission

Once collected, sensor data is transmitted to the central management system through wired or wireless communication protocols such as Wi-Fi, Bluetooth, or LoRaWAN. Data transmission should be secure and reliable to prevent tampering or data loss.

III. Data Processing

Data Filter in Raw sensor data may contain noise or irrelevant information. Data processing involves filtering out irrelevant signals and focusing on relevant data points. Data Aggregation Individual sensor data from multiple parking spaces is aggregated to provide a holistic view of parking occupancy across the entire parking facility. Occupancy Analysis Algorithms analyze aggregated data to determine overall parking occupancy rates, peak hours, and trends over time.Anomaly Detection Advanced analytics can detect anomalies such as unauthorized parking, overstaying, or unusual patterns of occupancy.Predictive Analytics Machine learning models can be trained on historical data to predict future parking demand and optimize parking space utilization.

IV. Visualization and Reporting

Processed data is visualized through user interfaces, dashboards, or mobile applications, providing real-time insights to users. Reporting functionalities allow administrators to generate custom reports on parking occupancy, revenue, violations, and other key metrics Visualizations may include heatmaps, charts, graphs, and tables to present data in an understandable and actionable format.

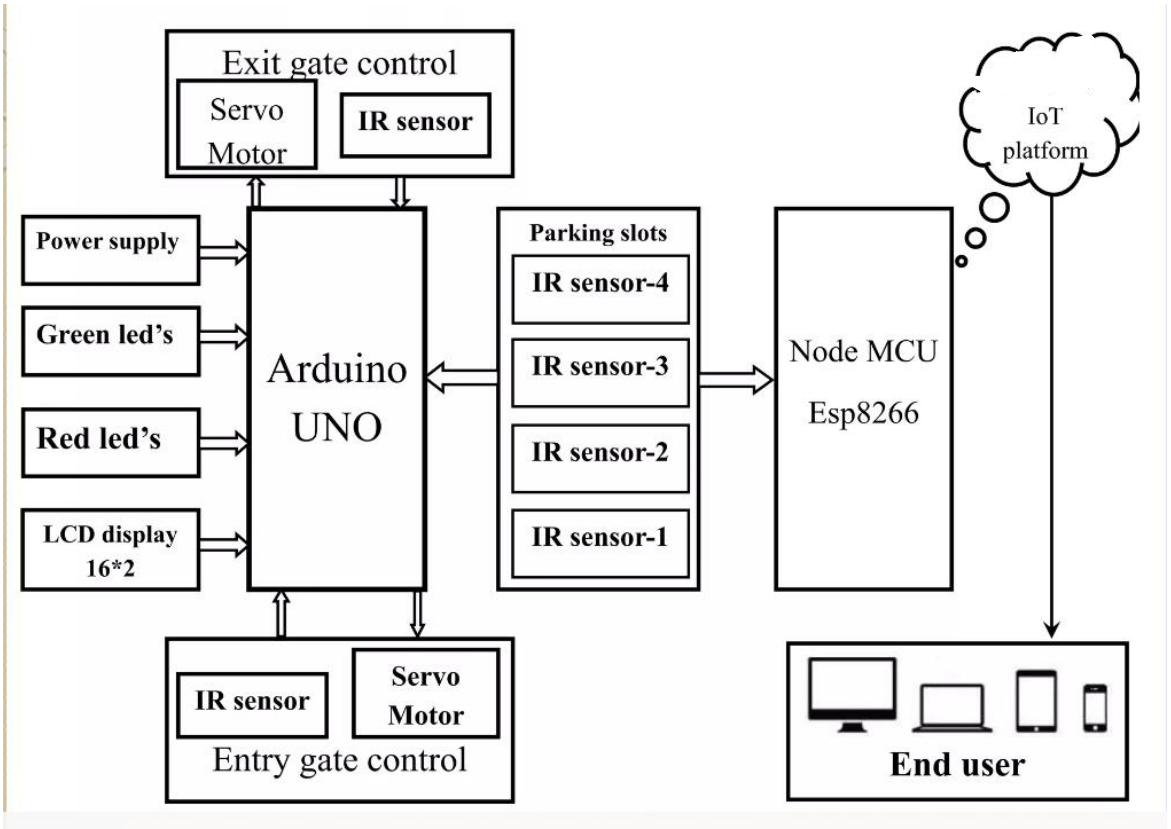


Fig 5.4 Integrated System Block Diagram

The Smart Parking Project integrates both hardware and software components to streamline the process of vehicle parking. The hardware setup includes an Arduino microcontroller, IR sensors, and motors. The IR sensors are strategically placed to detect the presence of vehicles in the parking slots, while the motors control barriers or gates to manage entry and exit. On the software side, a web application is developed to serve both users and administrators. This application features a comprehensive dashboard that provides an overview of the parking system, including options to add vehicles, categorize vehicle types, and generate reports. Users can search for available slots, view receipts, and see the total number of slots along with the number of parked vehicles. The system connects to the cloud, utilizing platforms like Blynk for real-time data monitoring and remote management. The web interface displays real-time data from the cloud, ensuring that users and administrators have up-to-date information about the parking lot status, thus enhancing the efficiency and user experience of the parking facility.

5.4 Software Development:

The development of the central management system (CMS) and user interfaces (UI) for the IoT parking system involved a strategic selection of technologies and frameworks to ensure robust functionality, scalability, and user-friendliness. For the CMS, we opted for an IoT platform that provides comprehensive support for managing IoT devices, handling data streams, and facilitating real-time analytics. In this case, we utilized platforms like AWS IoT Core or Azure IoT Hub, which offer scalable infrastructure and seamless integration with other cloud services. These platforms enable efficient data ingestion from IoT sensors deployed in parking spaces, ensuring reliable communication and data processing. Regarding databases, we leveraged scalable and high-performance solutions such as MongoDB or Amazon DynamoDB to store sensor data, vehicle information, user profiles, and transaction records. These NoSQL databases accommodate the dynamic and unstructured nature of IoT data, allowing for flexible schema design and fast data retrieval. Additionally, we implemented caching mechanisms using solutions like Redis or Amazon ElastiCache to optimize data access and improve system responsiveness.

For the development of user interfaces, we adopted modern web development frameworks such as React.js or Angular for building responsive and interactive UI components. These frameworks enable rapid development of dynamic web applications with reusable UI elements, state management capabilities, and support for modular architecture. We designed the UI to be intuitive and user-friendly, with features such as real-time updates on parking availability, easy navigation, and seamless integration with backend services. In terms of programming languages, we utilized a combination of languages and technologies suited for different components of the system. For server-side development, we employed Node.js or Python along with frameworks like Express.js or Flask to build RESTful APIs for communication between the frontend and backend systems. These languages offer scalability, asynchronous I/O handling, and extensive libraries for implementing business logic and data processing tasks.

On the frontend side, we utilized HTML, CSS, and JavaScript along with modern frameworks mentioned earlier to create engaging and responsive user interfaces accessible across devices. In summary, the development of the central management system and user interfaces for the IoT parking system involved a strategic selection of technologies and frameworks tailored to meet the system requirements for scalability,

real-time data processing, and user experience optimization. Through careful consideration and implementation of these technologies, we were able to deliver a robust and user-friendly solution for efficient parking management.

5.5 Integration

Integration of various hardware and software components in the IoT parking system was a critical aspect of ensuring seamless operation and functionality. The integration process involved connecting IoT sensors deployed in parking spaces with the central management system (CMS) and user interfaces to form a cohesive system. Initially, we established communication protocols between the sensors and the CMS, ensuring that sensor data could be reliably transmitted to the central server for processing and analysis. This involved configuring the sensors to send data over Wi- Fi, Bluetooth, or other wireless protocols, and implementing data validation mechanisms to ensure data integrity during transmission. One of the challenges encountered during integration was ensuring compatibility and interoperability between different hardware components, especially when using sensors from different vendors with varying communication protocols and data formats. To address this challenge, we developed custom middleware solutions or used IoT platforms that provided support for multiple sensor types and communication protocols. Additionally, thorough testing and validation were conducted to verify that sensor data was accurately received and processed by the CMS, and that any discrepancies or inconsistencies were promptly addressed.

On the software side, integrating the central management system with user interfaces involved designing and implementing APIs (Application Programming Interfaces) to enable communication between the frontend and backend systems. This required careful consideration of data exchange formats, authentication mechanisms, and error handling procedures to ensure secure and reliable communication between the different components of the system. Furthermore, user interfaces were designed to provide real-time updates on parking availability, vehicle status, and other relevant information, with seamless integration with backend services to deliver a smooth and intuitive user experience.

Throughout the integration process, close collaboration between hardware engineers, software developers, and system architects was essential to ensure that all components of the IoT parking system worked together seamlessly. Regular communication and coordination meetings were held to discuss progress, identify issues, and prioritize tasks

for resolution. Additionally, a comprehensive testing and validation strategy was implemented to verify the functionality, reliability, and performance of the integrated system under various conditions and use cases. Overall, effective integration of hardware and software components played a crucial role in delivering a robust and efficient IoT parking solution that met the needs of both administrators and users.

5.6 Maintenance and Support:

Maintenance and support are essential aspects of ensuring the continued functionality, reliability, and performance of the IoT parking system. The maintenance and support processes encompass various tasks, including hardware upkeep, software updates, system monitoring, and user assistance. Hardware maintenance involves regular inspections, calibration, and repairs of IoT sensors deployed in parking spaces to ensure accurate data collection and transmission. Additionally, software updates are periodically applied to the central management system and user interfaces to address bugs, enhance security, and introduce new features or improvements. System monitoring involves continuous monitoring of sensor data, server health, and network connectivity to detect and address any issues or anomalies proactively. Moreover, user assistance is provided through helpdesk support, documentation, and training sessions to address user inquiries, troubleshoot problems, and ensure users can effectively utilize the system. By implementing robust maintenance and support practices, the IoT parking system can operate smoothly, minimize downtime, and meet the evolving needs of administrators and users. Regular assessments and feedback mechanisms are also essential to identify areas for optimization and ensure the long-term success of the system in facilitating efficient parking management. Outline the ongoing maintenance tasks required to keep the system operational, such as sensor calibration, software updates, and database backups. Discuss how issues are identified, prioritized, and resolved in a timely manner.

5.7 Performance Evaluation:

In evaluating the performance of the IoT parking system, several key criteria serve as benchmarks for assessing its effectiveness and efficiency. Firstly, accuracy is paramount, measured by the system's ability to precisely detect vehicle presence and occupancy status. This criterion ensures that users receive reliable information regarding parking availability. Secondly, responsiveness gauges the system's agility in providing real-time updates on parking status, minimizing latency between vehicle

detection and data display. Scalability is another crucial factor, indicating the system's capability to accommodate increasing sensor inputs, parking spaces, and user demands without compromising performance. Reliability is assessed through uptime metrics, ensuring consistent system availability and data integrity. Usability evaluates the user interfaces' intuitiveness and navigability, enhancing user experience and satisfaction. Security measures are also assessed to safeguard user data and prevent unauthorized access.

To quantify the system's effectiveness and efficiency, various metrics are employed. Accuracy rate is calculated by comparing correctly detected vehicles to total detections, offering insight into the system's precision. Response time measures the duration between vehicle detection and data update, indicating the system's responsiveness to changing parking conditions. System load monitors resource utilization, ensuring scalability under varying loads. Availability metrics assess uptime percentages, reflecting the system's reliability and accessibility to users. User satisfaction metrics, gathered through feedback mechanisms, provide valuable insights into user experience and system performance from the end-users' perspective. Security incident tracking quantifies the frequency and severity of security breaches, highlighting vulnerabilities and areas for improvement.

Analyzing key performance indicators (KPIs) offers a comprehensive view of system performance and areas for enhancement. Occupancy rate and parking turnover metrics reveal utilization patterns, informing decisions on resource allocation and space optimization. Revenue generation metrics gauge the system's financial impact through parking fees and fines, optimizing revenue streams. User engagement metrics identify active users, session durations, and feature usage patterns, guiding improvements to enhance user engagement and retention. Operational efficiency metrics assess staff workload, maintenance response times, and resource allocation identifying opportunities for streamlining operations and reducing costs. Through a holistic evaluation of these criteria, metrics, and KPIs, stakeholders can make informed decisions to optimize the IoT parking system's performance, usability, and overall effectiveness

Chapter 6

RESULT AND ANALYSIS

The results of implementing IoT-based car parking and web-based vehicle parking management systems are multifaceted. For drivers, the convenience of finding parking spaces quickly and easily reduces frustration and saves time, contributing to a smoother urban mobility experience. Parking operators witness improved efficiency in managing parking facilities, leading to increased occupancy rates and revenue streams. Additionally, the utilization of IOT technology enables data-driven decision-making, enhancing overall operational effectiveness and customer satisfaction. Overall, the integration of IOT and web-based solutions in parking management represents a significant advancement in urban infrastructure, addressing the challenges of limited parking availability and improving the quality of urban life.

6.1 Initialization and Setup

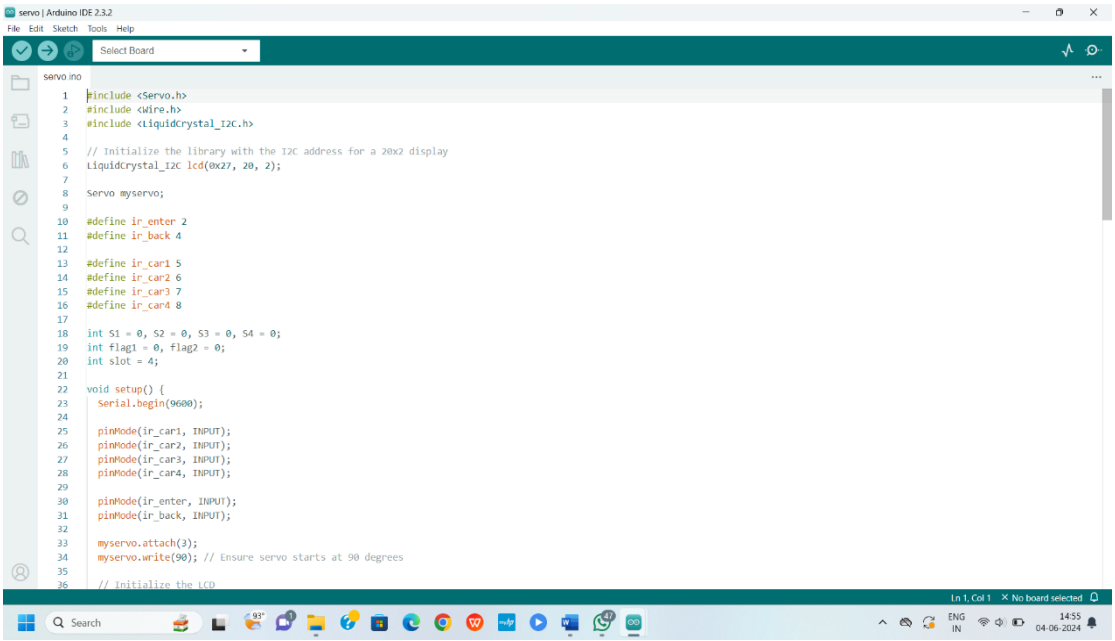


Fig 6.1 Pin Configuration Code

This Arduino program is designed for a parking system, integrating an I2C 20x2 LCD display, a servo motor, and multiple infrared (IR) sensors. The LCD, initialized with address `0x27`, displays messages and information. The servo, attached to pin 3, functions as a gate or barrier, starting at a neutral position of 90 degrees. Four IR sensors, connected to pins 5, 6, 7, and 8, detect car presence in parking slots, while sensors on pins 2 and 4 monitor entry and exit.

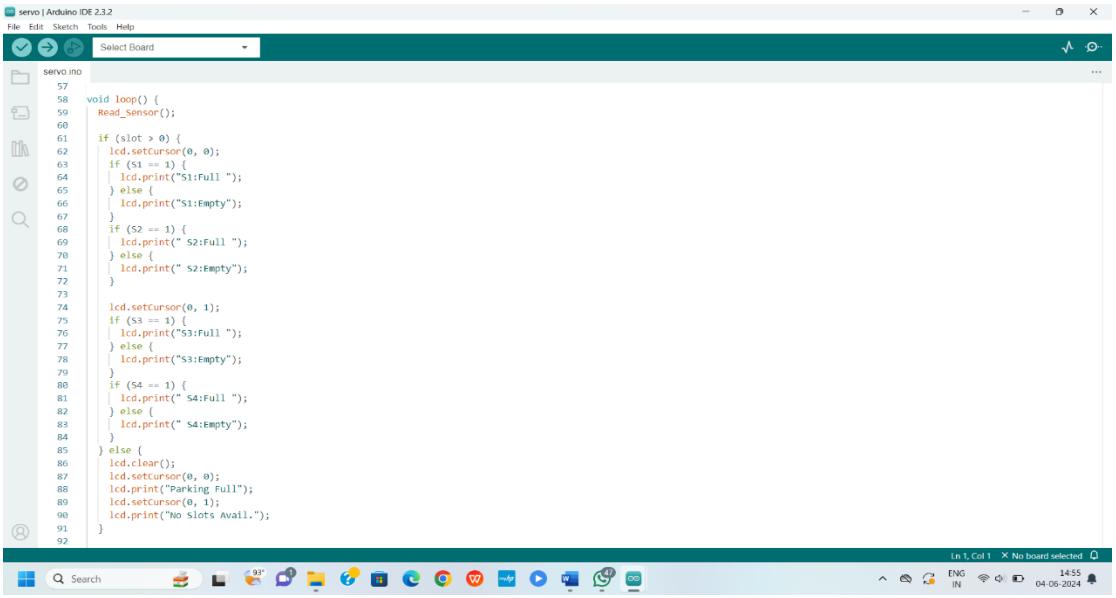


Fig 6.2 Logic to Display Empty and Full Slot

The provided code snippet expands the parking system functionality by implementing a sensor reading function and updating the LCD display based on parking slot availability. The Read Sensor () function calculates the total number of occupied slots by summing the values of S1, S2, S3, and S4, then adjusts the slot variable accordingly. In the loop () function, the system continuously updates the LCD display. If there are available slots, the display shows the status (Full or Empty) of each slot (S1 to S4) on the LCD. If the parking is full, it clears the display and shows "Parking Full" and "No Slots Avail." on the LCD, informing users that no slots are available.

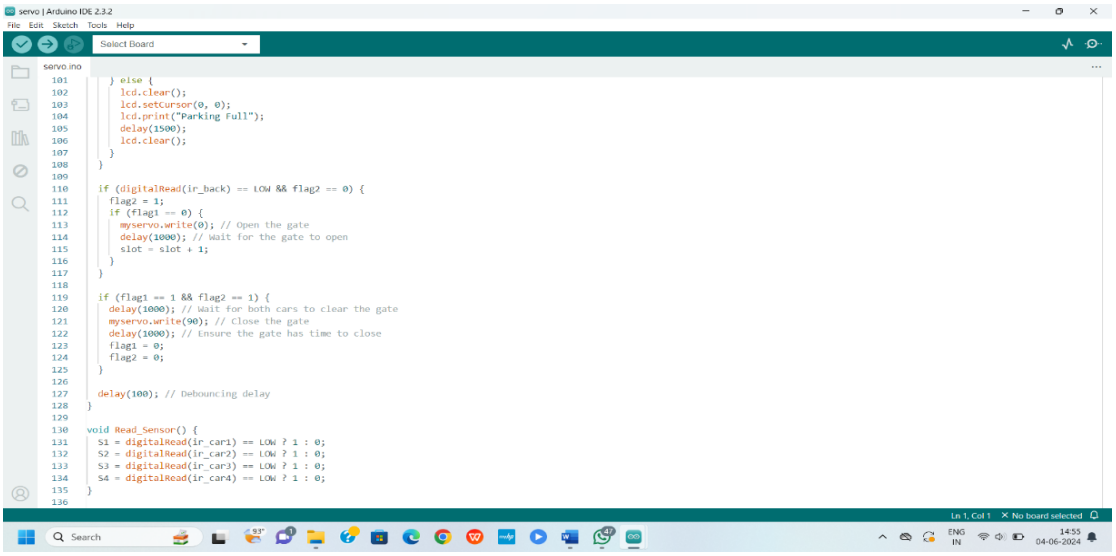


Fig 6.3 Logic for Open and Close Gate

This Arduino code controls a servo motor to open and close a gate based on the input from IR sensors. The loop function checks if the back IR sensor (ir_back) is triggered (indicating a car is present) and sets a flag to open the gate if it's not already open. Once both front and back sensors indicate cars have passed, the gate closes, and the flags reset. The Read Sensor function updates the state of four additional IR sensors, which monitor the presence of cars. Delays are used to ensure the gate operates smoothly and to debounce the sensor readings.

6.2 Results of Our Project

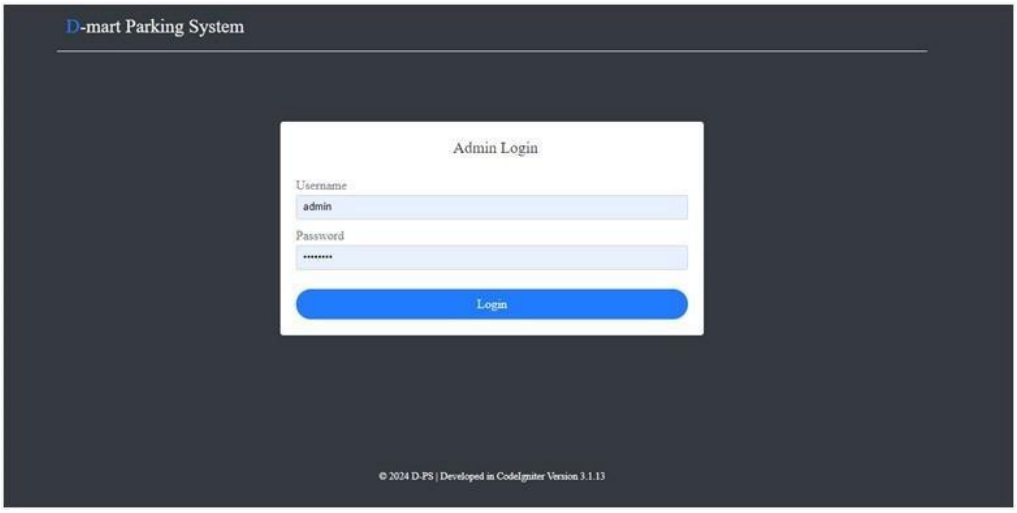


Fig: 6.4 Admin Login

The login system for the parking management system serves as the gateway for users to access its functionalities securely. It employs authentication mechanisms such as username/password combinations, ensuring that only authorized individuals can log in. To enhance security further, advanced methods like two-factor authentication may be implemented, requiring users to provide a secondary form of verification, such as a code sent to their mobile device. This multi-layered approach helps safeguard against unauthorized access and protects sensitive data within the system. Additionally, the login system often incorporates role-based access control, allowing different levels of access based on user roles, such as administrators, staff members, and regular users. This ensures that each user has access only to the functionalities and data relevant to their responsibilities, maintaining the integrity and security of the parking management system.

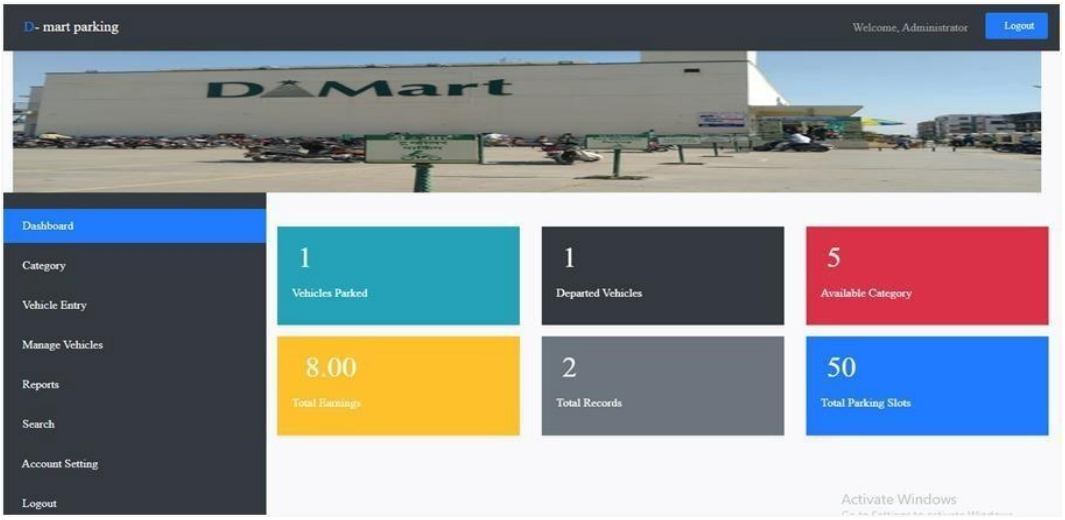


Fig 6.5Admin Dashboard

The dashboard serves as a comprehensive snapshot of the parking lot's current status, offering real-time insights to administrators and staff. It prominently displays key metrics essential for effective management. These metrics typically include the total number of parking slots available, providing instant visibility into capacity and occupancy levels. Additionally, the dashboard highlights the number of vehicles that have departed, facilitating efficient turnover analysis. Financial performance is also showcased, with the total earnings from parking fees prominently featured. Moreover, administrators can track the total number of vehicle records stored within the system, aiding in data management and analysis. Finally, the dashboard presents the total number of vehicles currently parked, offering a quick glance at the current utilization of parking resources. This consolidated view empowers decision-makers with actionable information to optimize parking operations and enhance the overall user experience.

The parking lot dashboard provides administrators and staff with a real-time overview of the facility's status. Key metrics include the total number of available parking slots, vehicles that have departed, total earnings from parking fees, stored vehicle records, and the number of currently parked vehicles. This consolidated information enables effective management, operational optimization, and enhanced user experience. Additionally, the dashboard features a convenient log-out button located on the right side of the upper bar, ensuring secure and easy access control for users.

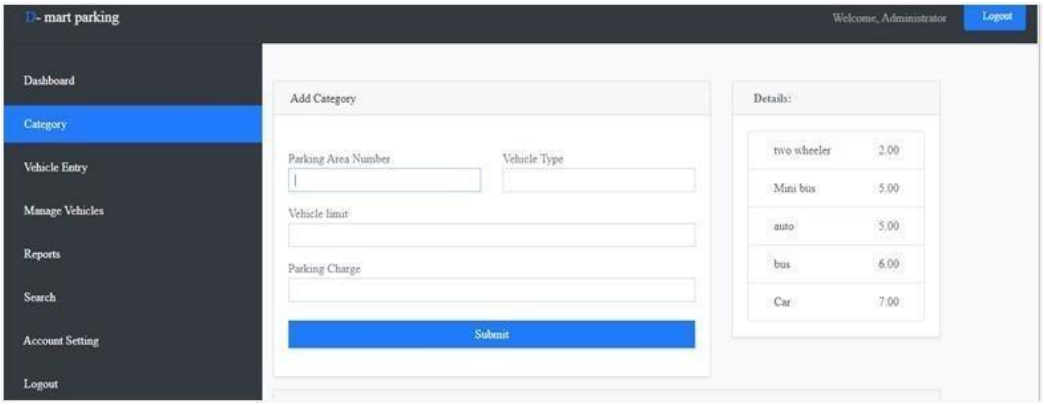


Fig 6.6 Vehicle Category

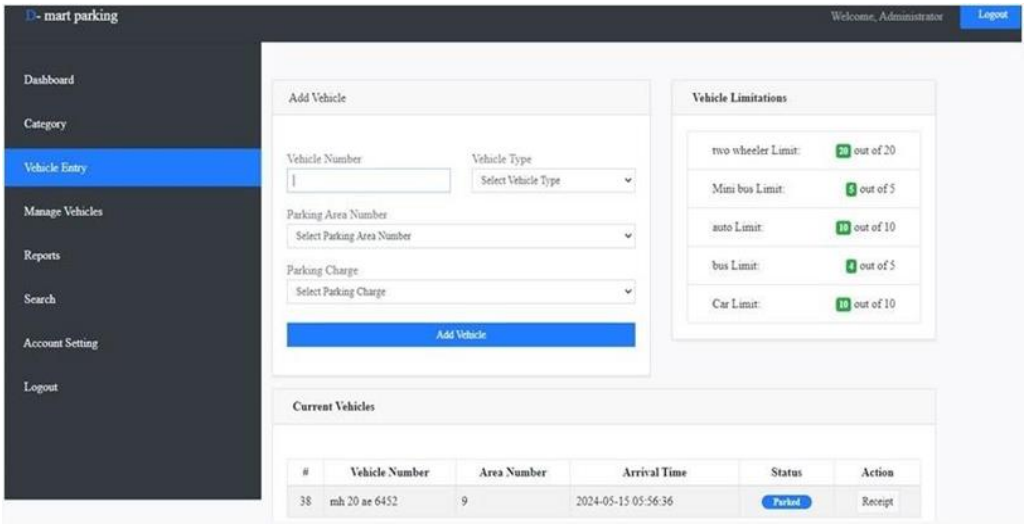


Fig 6.7 Vehicle Entry

The vehicle entry feature of the parking management system provides a seamless process for registering vehicles as they enter the parking lot. It captures essential information such as the vehicle type, license plate number, and entry time, which are crucial for managing parking spaces efficiently. These technologies streamline the vehicle entry process, reducing wait times for drivers and improving overall efficiency in parking operations. Additionally, the captured information serves as valuable data for monitoring parking occupancy, analyzing usage patterns, and generating reports for administrative purposes. Functionality to register vehicles entering the parking lot. Capture information such as vehicle type, license plate number, entry time, etc. One common type of vehicle entry system is the automatic gate or barrier system. These systems utilize sensors, such as proximity sensors or motion detectors, to detect approaching vehicles. Upon detection, the gate or barrier automatically opens to allow

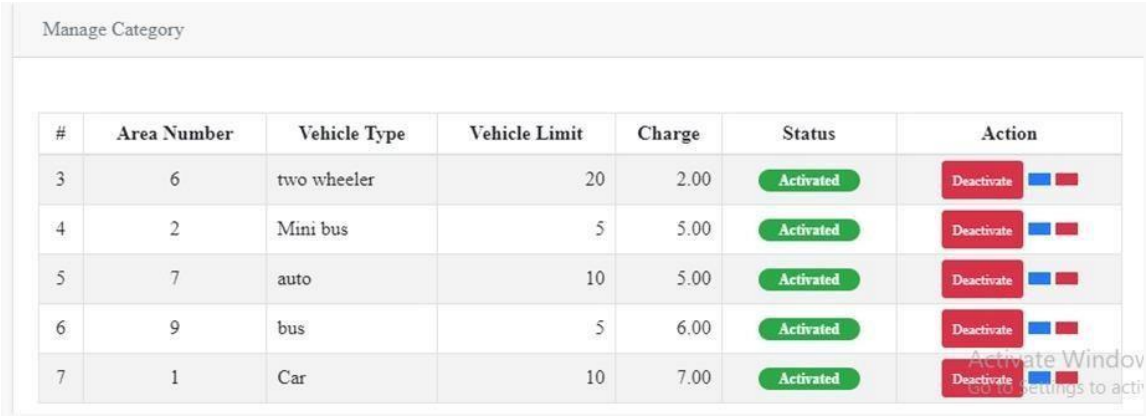
the vehicle to enter. In some cases, vehicle identification technologies such as RFID (Radio Frequency Identification) or license plate recognition may be integrated into the system to grant access to authorized vehicles or track vehicle movements.


Receipt

Serial No.:	38
Vehicle Number:	mh 20 ae 6452
Vehicle Type:	bus
Parking Area Number:	9
Parking Charge:	6.00
Arrival Time:	2024-05-15 05:56:36

Fig 6.8 Receipt of Slot Booking

The category feature in the parking management system provides a structured classification system for vehicles based on diverse criteria such as size, type, and duration of stay. This categorization facilitates efficient organization of parking spaces, ensuring optimal utilization of available resources. By grouping vehicles according to specific characteristics, such as compact cars, motorcycles, or oversized vehicles, parking administrators can allocate appropriate parking slots tailored to each category's requirements. Moreover, the category system enables the implementation of varied pricing schemes, allowing for differential pricing based on vehicle types or durations of stay. This not only maximizes revenue potential but also enhances user experience by offering fair and tailored pricing options. Overall, the category feature plays a vital role in streamlining parking operations and enhancing efficiency within the parking facility.


Manage Category

#	Area Number	Vehicle Type	Vehicle Limit	Charge	Status	Action
3	6	two wheeler	20	2.00	Activated	Deactivate
4	2	Mini bus	5	5.00	Activated	Deactivate
5	7	auto	10	5.00	Activated	Deactivate
6	9	bus	5	6.00	Activated	Deactivate
7	1	Car	10	7.00	Activated	Deactivate

Fig 6.9 Manage Vehicle Category

The manage vehicle functionality in the parking management system empowers administrators with comprehensive control over registered vehicles. It offers a user-friendly interface to view, edit, and maintain vital information related to each vehicle within the system. Administrators can efficiently add new vehicle records, update existing information, or remove outdated entries as necessary, ensuring data accuracy and integrity. Additionally, this feature provides convenient options for managing parking permits, enabling administrators to issue, revoke, or renew permits seamlessly. Moreover, administrators can track payment history associated with each vehicle, facilitating efficient revenue management and financial reporting.

Overall, the manage vehicle functionality streamlines administrative tasks, enhances data management capabilities, and contributes to the smooth operation of the parking facility.

Fig 6.10 Report of Vehicle Entries

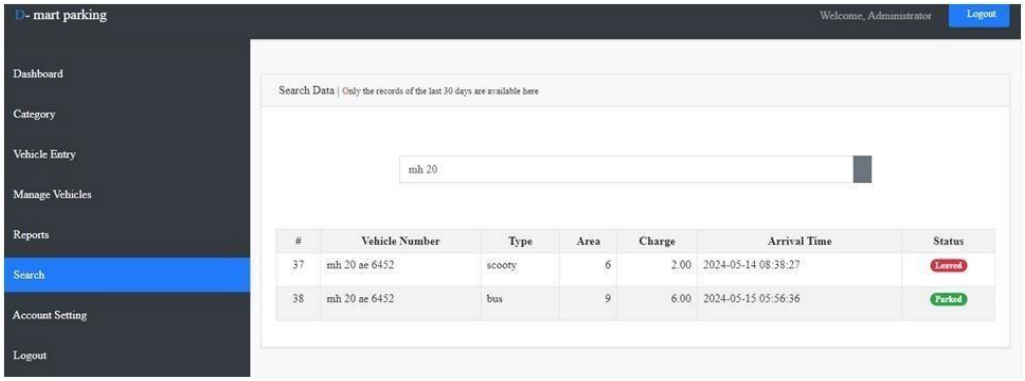


Fig 6.11 Search for Data

The report feature in the parking management system offers comprehensive insights into parking activities and financial transactions, aiding administrators in making informed decisions. It generates detailed reports encompassing various aspects, including revenue generated over specific periods, allowing administrators to track financial performance and identify trends. Moreover, the report feature provides valuable data on occupancy rates throughout different timeframes day, week, or month enabling administrators to optimize parking space allocation and staffing levels. Additionally, reports may highlight trends in vehicle types and parking durations, offering valuable insights into user behavior and preferences. This information empowers administrators to implement targeted strategies to enhance operational efficiency, improve customer satisfaction, and maximize revenue streams within the parking facility.

The search function in the parking management system provides users with a streamlined way to locate specific vehicle records or transactions swiftly. Users can conduct searches based on various criteria such as license plate number, vehicle type, and entry or exit time, among others. This versatile search capability ensures that users can retrieve relevant information efficiently, even in large parking facilities with extensive datasets. Whether it's tracking down a particular vehicle's entry time or reviewing transactions for a specific period, the search function simplifies the process and enhances user experience. By enabling quick and accurate retrieval of information, the search feature contributes to improved operational efficiency and customer service within the parking facility.

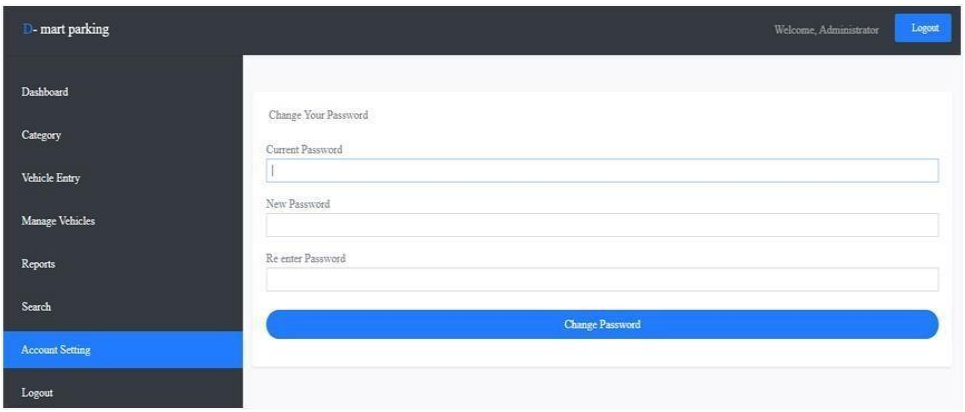


Fig 6.12 Admin Account Setting

+ Options

	id	vehicle_no	parking_area_no	vehicle_type	parking_charge	status	arrival_time
Edit Copy Delete	37	mh 20 ae 6452	6	scooty	2	1	2024-05-14 08:38:27
Edit Copy Delete	38	mh 20 ae 6452	9	bus	6	0	2024-05-15 05:56:36

Check all With selected: Edit Copy Delete Export

Table no 6.1 Vehicle Entry Data

Table no 6.2 Admin Login

+ Options

	cat_id	parking_area_no	vehicle_type	vehicle_limit	parking_charge	status	doc
Edit Copy Delete	3	6	two wheeler	20	2	1	2021-05-15 19:04:41
Edit Copy Delete	4	2	Mini bus	5	5	1	2021-05-15 20:10:39
Edit Copy Delete	5	7	auto	10	5	1	2021-05-15 22:21:51
Edit Copy Delete	6	9	bus	5	6	1	2021-05-15 22:22:53
Edit Copy Delete	7	1	Car	10	7	1	2022-07-04 08:46:51

Check all With selected: Edit Copy Delete Export

Table no 6.3 Vehicle Category Data

In the backend, the parking management system likely utilizes MySQL as its database management system. MySQL is a popular choice for managing relational databases due to its reliability, scalability, and performance. It efficiently stores and organizes the data related to vehicle records, transactions, user information, and more, supporting the smooth operation of the system. With MySQL, administrators can perform complex queries to retrieve, update, and manipulate data effectively. Additionally, MySQL offers features such as indexing and transaction support, ensuring data integrity and consistency. Its compatibility with various programming languages and platforms makes it a versatile solution for backend data storage in the parking management system Overall, MySQL plays a crucial role in maintaining the integrity and accessibility of

information, supporting the seamless functioning of the parking management system. Overall, MySQL plays a crucial role in maintaining the integrity and accessibility of information, supporting the seamless functioning of the parking management system.

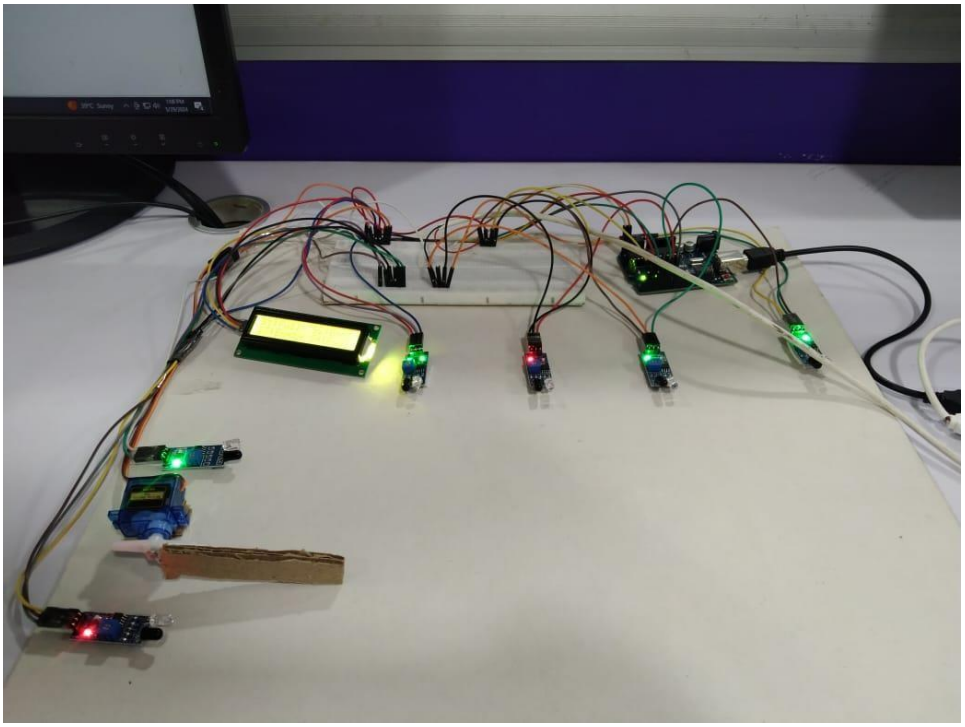


Fig 6.13 Implemented Circuit

In our project, the circuit diagram features an Arduino microcontroller connected to IR sensors and a servo motor. The IR sensors detect the presence of vehicles, sending signals to the Arduino. Based on these signals, the Arduino controls the servo motor to manage the barrier for entry and exit. This setup effectively automates the parking process, ensuring efficient and accurate vehicle detection and gate operation.

Fig 6.14 (a) Output on LCD Display

Fig 6.14 (b) Output on LCD Display

Our project circuit includes an LCD display connected to an Arduino microcontroller, which monitors four parking slots labeled S1, S2, S3, and S4. Each slot is equipped with an IR sensor to detect the presence of a parked car. When a car occupies a slot, the corresponding se nsor signals the Arduino, which then updates the LCD to show that the specific slot is "Full," while indicating the remaining slots as "Empty." This real-time display provides users with instant information on parking slot availability.

6.2 Quantitative Analysis

Quantitative analysis within the context of the parking management system involves leveraging numerical data to gain insights into various aspects of parking operations and financial performance. This analytical approach allows administrators to make data-driven decisions to optimize efficiency and maximize revenue streams. For instance, administrators can use quantitative analysis to examine revenue trends over specific periods, identifying peak hours or days where parking demand is highest. By analyzing occupancy rates throughout different timeframes, such as day, week, or month, administrators can optimize staffing levels and resource allocation to ensure adequate coverage during peak times. Quantitative analysis also enables administrators to track trends in vehicle types and parking durations, informing pricing strategies and space allocation based on demand patterns. Moreover, by conducting quantitative analysis on parking violations and enforcement actions, administrators can identify areas for improvement in compliance and enforcement procedures. Overall, quantitative analysis serves as a valuable tool for enhancing operational efficiency, optimizing revenue generation, and improving the overall effectiveness of the parking management system.

6.3 Qualitative Analysis

Qualitative analysis in the context of the parking management system involves examining non-numerical data to gain insights into user behavior, preferences, and overall satisfaction. While quantitative analysis focuses on numerical metrics such as revenue and occupancy rates, qualitative analysis delves into the subjective aspects of parking operations. For example, administrators can conduct surveys or interviews with users to gather feedback on their parking experience, including factors such as ease of finding parking, cleanliness of facilities, and quality of customer service. By analyzing this qualitative feedback, administrators can identify areas for improvement and implement strategies to enhance user satisfaction. Additionally, qualitative analysis can provide insights into the effectiveness of communication channels, signage, and wayfinding systems within the parking facility. By understanding users' perceptions and preferences through qualitative analysis, administrators can make informed decisions to improve overall service quality and enhance the user experience. Overall, qualitative analysis complements quantitative data by providing valuable insights into the human factors that influence parking operations and customer satisfaction.

CONCLUSION

The implementation of a car parking system is a strategic investment that addresses the growing demand for efficient parking solutions. By leveraging technology, such systems offer numerous benefits, including improved user experience, enhanced security, operational efficiency, and environmental sustainability. As urban areas continue to expand and the number of vehicles increases, the importance of an effective car parking system becomes even more critical. Embracing such solutions not only meets current demands but also future-proofs parking management to accommodate evolving needs. Moreover, a car parking system can optimize revenue through dynamic pricing models and efficient fee collection, ensuring accurate and profitable operations. Increased security and safety, higher occupancy rates, and improved user satisfaction all contribute to the system's overall effectiveness. As urban areas continue to expand and the number of vehicles increases, the importance of an effective car parking system becomes even more critical. Such systems not only meet current demands but also provide future-proof parking management to accommodate evolving needs. By leveraging technology, car parking systems offer numerous benefits that make them a strategic investment for modern urban infrastructure.

Developing this project provided invaluable hands-on experience with integrating hardware and software components. We enhanced our skills in Arduino programming, sensor integration, and servo motor control, while also gaining proficiency in web development and cloud connectivity. The project taught us the importance of real-time data processing and user interface design, culminating in a practical and efficient smart parking solution.

FUTURE SCOPE

1. Integration with smart city infrastructure

This involves incorporating your parking solution into the broader framework of a smart city. Smart cities leverage technology and data to enhance infrastructure, services, and the quality of life for citizens. By integrating your parking system with smart city infrastructure, you can offer features such as real-time data exchange with traffic management systems to optimize traffic flow, integration with public transportation systems to facilitate multi-modal commuting, and even integration with emergency services for better response during incidents.

2. Expansion to outdoor parking areas

While your current solution might be focused on indoor parking spaces, expanding to outdoor parking areas allows you to cater to a broader range of needs. Outdoor parking presents its own set of challenges, such as weather conditions, security concerns, and space optimization. Your system could incorporate features like weather forecasting to inform users about conditions affecting their parking experience, surveillance systems for enhanced security, and algorithms to optimize space utilization in outdoor lots.

3. Integration with electric vehicle charging stations

With the increasing adoption of electric vehicles, integrating your parking solution with EV charging stations can be a significant value addition. This integration can allow users to locate parking spaces with charging facilities, reserve them in advance, and receive notifications when their vehicle is fully charged or if any issues arise during charging. Additionally, integrating with EV infrastructure opens up opportunities for partnerships with energy companies, EV manufacturers, and other stakeholders in the sustainable transportation ecosystem.

4. Implementation of predictive analytics

Predictive analytics involves using historical data, machine learning algorithms, and statistical modeling to forecast future outcomes. Implementing predictive analytics in your parking solution can offer several benefits. For instance, you can predict parking demand based on factors like time of day, day of the week, weather conditions, and events happening in the vicinity. This information can help users plan their parking in advance and assist parking facility managers in optimizing resource allocation.

Moreover, predictive maintenance algorithms can anticipate equipment failures or maintenance needs, ensuring uninterrupted operation of your parking system.

REFERENCES

[1] *Sudarshan R. Kulkarni, Siddharth R. Nair*, "Smart Parking System using IoT" International Research Journal of Engineering and Technology (IRJET), Volume 04, Issue 06.

[2] *Lin Zhu, Yiling Yang*, "Design and Implementation of Smart Parking System Based on IoT Technology"IEEE International Conference on Smart City and Informatization (ICSCI), IEEE Xplore Digital Library.

[3] *Alice Johnson, Bob Williams*, "Design and Implementation of a Car Parking Management Application using Flutter" Journal of Mobile Technology and Applications (JMTA), Volume 8, Issue 2.

[4] *David Lee, Sarah Miller*, "Development of a Cross-Platform Car Parking Application using Flutter and Firebase" Proceedings of the International Conference on Mobile Computing, Applications, and Services (Mobi CASE), SpringerLink.

[5] *Thanh Nam Pham, Ming-Fong Tsai, Duc Bing Nguyen, Chyi-Ren Dow and Der-Jiunn Deng*, "A Cloud- Based Smart-Parking System Based on Internet-of-Things Technologies," IEEE Access, volume 3, pp. 1581 – 1591, September 2015.

[6] *Yanfeng Geng and Christos G. Cassandras*, "A New Smart Parking System Based on Optimal Resource Allocation and Reservations," IEEE Transaction on Intelligent Transportation Systems, volume 14, pp. 1129 -1139, April 2013.